

Foreman: единое рабочее руководство

Веб-браузер, мобильное приложение, Telegram-бот

PDF-версия сделана для спокойного чтения: обязательные главы, справочные места и администраторские разделы отмечены отдельно.



Сформировано из актуального Markdown-руководства.

Как читать PDF

Руководство большое, поэтому его не нужно читать подряд от первой до последней страницы. Метки помогают быстро понять, кому нужен раздел.

Читать всем

Базовые главы для прорабов, офиса и руководителя.

Практика

Готовые сценарии и частые проблемы.

По необходимости

Разделы, которые открывают, когда появляется конкретная задача.

Справочно

Фоновые объяснения: безопасность, проверки, ограничения.

Администратору

Команды и действия, которые нужны только ответственному администратору.

Оглавление

	Foreman: единое рабочее руководство	6
Читать всем	Веб-браузер, мобильное приложение, Telegram-бот и Web Push	6
Для чтения	Быстро попробовать Foreman	6
Как пользоваться	Как читать это руководство	7
Читать всем	0. Короткая карта системы	8
Для чтения	0.1 Что где делать	8
Для чтения	0.2 Как связаны три направления	9
Справочно	0.3 Где хранится информация: PostgreSQL <code>foreman-db</code>	9
Читать всем	0.4 Почему это важно для прорабов и офиса	11
Справочно	0.5 Как Foreman проверяет файлы	11
Справочно	0.6 Как Foreman читает заявки через распознавание файлов и <code>office-converter</code>	15
Справочно	0.7 Как Foreman проверяется перед выпуском в работу	17
Читать всем	Перед началом: пользователи, вход и права	30
Читать всем	B.1 Вход в веб-браузере	30
Читать всем	B.2 Вход в мобильном приложении	30
Читать всем	B.3 Сколько держится вход	31
Для чтения	B.4 Роль и должность	31
Для чтения	B.5 Основные роли	31
Для чтения	B.6 Гостевой наблюдатель	31
Справочно	B.7 Безопасность Foreman: общие правила	33

Читать всем	B.8 Безопасность в веб-браузере Foreman Web	39
Справочно	B.9 Безопасность в мобильном приложении Foreman	44
Читать всем	B.10 Где смотреть безопасность Telegram-бота	46
Читать всем	Раздел 1. Foreman Web в браузере	47
Для чтения	1.1 Верхняя панель	47
Для чтения	1.2 Левая колонка в журнале	47
Для чтения	1.3 Как открыть заявку	48
Для чтения	1.4 Кнопки в карточке заявки	48
Для чтения	1.5 Preview, PDF, DOCX и отправка на e-mail	48
Для чтения	1.6 Создание новой заявки в вебе	49
Для чтения	1.7 Черновики в вебе	49
Для чтения	1.8 Вложения в заявке	49
Для чтения	1.9 Как выглядят вложения	50
Для чтения	1.10 Внешние вложения	50
Для чтения	1.11 Журнал доставки	51
По необходимости	1.12 Документохранилище	51
По необходимости	1.13 Таймлайн	55
По необходимости	1.14 План-график	56
Администратору	1.15 Профиль и администрирование	60
Читать всем	Раздел 2. Мобильная версия Foreman	66
Для чтения	2.1 Главный экран	66
Читать всем	2.2 Вход	67
Для чтения	2.3 Восстановить доступ	68
Для чтения	2.4 Активировать доступ	68
Для чтения	2.5 Обновление приложения	69
Для чтения	2.6 Заявка на заполнения	69
Для чтения	2.7 Заявка на расходники	70
Для чтения	2.8 Балконный/оконный блок	70
Для чтения	2.9 Переделка рамы/створки	71
Для чтения	2.10 Журнал заявок в мобильном	71
Для чтения	2.11 Черновики и связь	72
Для чтения	2.12 Сервер и связь	72
Для чтения	2.13 Чего в мобильной версии нет	72
Читать всем	Раздел 3. Telegram-бот	73
Читать всем	3.1 Роли и права	73

Для чтения	3.2 Меню, помощь и кнопки	74
Для чтения	3.3 Личный и общий кабинет	74
Для чтения	3.4 Поиск заявок	75
Для чтения	3.5 Таблица учета	76
Для чтения	3.6 Файлы заявок	78
Читать всем	3.7 Карточка заявки в Telegram	79
Для чтения	3.8 Правка заявок	80
Читать всем	3.9 Создание заявок через Telegram-бота	81
Для чтения	3.10 Удаление и восстановление	82
Для чтения	3.11 Пересылка на e-mail	83
Для чтения	3.12 Отгрузка на объект	83
По необходимости	3.13 Реестр запусков	84
По необходимости	3.14 Шаблоны для замеров	84
По необходимости	3.15 Голосовой режим	85
По необходимости	3.16 Многосоставные команды	86
Для чтения	3.17 Личное общение	87
Администратору	3.18 Админские команды	87
Справочно	3.19 Устойчивость к паузам и ошибкам	87
Для чтения	3.20 Короткий словарь рабочих команд	93
Для чтения	3.21 Что осталось в отдельном README бота	94
Практика	Приложение А. Сквозные рабочие сценарии	94
Для чтения	A.1 Прораб создает заявку на объекте	95
Для чтения	A.2 Менеджер проверяет заявку на ПК	95
Для чтения	A.3 Прораб прикрепляет файл, который уже есть в Foreman	95
Для чтения	A.4 Руководитель смотрит движение заявок	95
По необходимости	A.5 Команда работает с план-графиком	95
Для чтения	A.6 Гость смотрит систему	95
Читать всем	A.7 Быстро найти файл через Telegram	96
Администратору	A.8 Администратор заводит нового сотрудника	96
Практика	Приложение В. Частые проблемы и решения	96
Для чтения	B.1 Белый экран в браузере	97
Для чтения	B.2 Приходится снова вводить пароль	97
Для чтения	B.3 Неактивная кнопка	97
Для чтения	B.4 Файл прикрепился некрасиво или не видно название	97
Читать всем	B.5 Мобильная заявка не отправляется	98

Читать всем	B.6 Telegram-бот отвечает не туда	98
Администратору	Приложение С. Администраторская памятка перед внедрением	98
Практика	Приложение D. Как объяснить сотруднику без давления	98
Администратору	Приложение E. Что считать главным документом	99

Foreman: единое рабочее руководство

Читая всем Веб-браузер, мобильное приложение, Telegram-бот и Web Push

Версия документа: 21.06.2026, редакция 7

Версионный PDF-файл для передачи и скачивания: [ЕДИНОЕ_РУКОВОДСТВО_FOREMAN_2026-06-21.pdf](#)

Что изменено в редакции 7: в руководство добавлен новый контур распознавания заявок из файлов. Простыми словами объяснено, что такое OCR, как Foreman через API использует Яндекс Vision OCR, зачем нужен отдельный `office-converter` для старых Word/Excel-файлов и что затем делает собственный parser Foreman. Обновлено тестовые числа после усиления этого контура: основной мега-тест теперь включает OCR release gate, а мератест-2 проверяет OCR mutation inventory и при необходимости запускает полный OCR mutation pilot. Формулировки оставлены честными: распознавание уже помогает читать заявки и замечать пропуски, но не отменяет ручную сверку сложных документов и включается в production только через controlled pilot.

Это главный документ по Foreman. Он заменяет устные объяснения вида “там нажми кнопку” и собирает три направления в одну рабочую картину. В обычной работе открывайте основной веб-адрес на домене `claim-bot`.

1. `Foreman Web` - работа в веб-браузере. 2. `Foreman Mobile` - мобильное приложение Foreman (устанавливается на смартфон с ОС Android). 3. `Telegram-бот` - быстрые команды, личный кабинет и чат-сценарии. 4. `Web Push` - короткие браузерные уведомления по выбранным событиям Foreman.

Для чтения Быстро попробовать Foreman

Основной веб-адрес для сотрудников: <https://claim-bot-tzertis.amvera.io/foreman/>

Диагностика сайта: <https://foreman-api-tzertis.amvera.io/foreman/>

Если хотите продолжить читать руководство и одновременно открыть ссылку, нажмите и удерживайте клавишу `Ctrl`, затем щелкните по ссылке левой кнопкой мыши. Ссылка откроется в отдельной вкладке, а руководство останется открытым.

Гостевой вход для ознакомления:

Поле	Значение
Логин	<code>vrem.prov</code>
Пароль	<code>vrem.prov</code>
Режим	только просмотр, без создания и изменения заявок

Гостевой режим подходит для первого знакомства: можно открыть журнал, карточки, таймлайн и общие документы, но нельзя менять рабочие данные.

Документ написан как инструкция, а не как реклама. Каждый раздел отвечает на вопросы:

- где находится действие;
- что нажать;
- что получится;
- кто имеет право;
- где типичная ошибка;
- что делать, если что-то пошло не так.

Как пользоваться

Как читать это руководство

Откройте тот раздел, который нужен прямо сейчас:

Раздел	Что внутри
Раздел 0. Короткая карта системы	Как связаны веб, мобильное приложение, Telegram-бот, база данных, файлы, OCR, безопасность и проверки перед выпуском
Раздел 1. Foreman Web в браузере	Журнал, карточки заявок, вложения, документохранилище, сортировки, таймлайн, план-график, профиль, Web Push, администрирование
Раздел 2. Мобильная версия Foreman	Вход с телефона, главный экран, заявки, черновики, фото, обновления, связь
Раздел 3. Telegram-бот	Меню, личный кабинет, поиск заявок и файлов, карточка, правка, удаление, e-mail, отгрузка, реестр, замеры, голос, админ-команды

После трех основных разделов идут приложения: готовые рабочие сценарии, частые проблемы, администраторская памятка и короткий текст для спокойного внедрения.

Читать всем

0. Короткая карта системы

Для чтения

0.1 Что где делать

Задача	Лучше делать в вебе	Лучше делать в мобильном	Лучше делать в Telegram
Смотреть журнал заявок	Да	Можно	Можно поиском
Открывать карточку заявки	Да	Да	Можно через карточку бота
Создавать заявку с объекта	Можно	Да	Можно в старых конструкторах
Создавать заявку спокойно на ПК	Да	Можно	Не лучший вариант
Прикреплять файлы с ПК	Да	Частично	Через команды бота
Прикреплять файлы из документохранилища	Да	Нет	Нет
Хранить общие документы	Да	Нет	Нет
Сортировать документы	Да	Нет	Нет
Смотреть таймлайн	Да	Нет	Нет
Скачать PDF таймлайна	Да	Нет	Нет
Работать с план-графиком	Да	Нет	Нет
Быстро найти заявку по номеру	Да	Да	Да
Быстро найти файл командой	Нет	Нет	Да
Работать голосом	Нет	Нет	Да
Админские проверки бота	Нет	Нет	Да

0.2 Как связаны три направления

Пользователь

- > Foreman Web в браузере
- > Foreman Mobile на телефоне
- > Telegram-бот

Все каналы обращаются к общей системе Foreman:

- > пользователи
- > заявки
- > файлы
- > антивирусный контур файлов
- > security monitoring и тревоги
- > документы
- > статусы
- > журнал
- > таймлайн
- > web push-уведомления

Главное правило: заявка должна жить в Foreman, а не в одиночном Word-файле на домашнем компьютере.

0.3 Где хранится информация: PostgreSQL foreman-db

Foreman использует современное серверное хранилище данных - PostgreSQL, база `foreman-db`. Это важно понимать: Foreman не является "папкой с Word-файлами" и не держит рабочий процесс только на личных компьютерах пользователей.

Простыми словами: когда пользователь создает заявку, прикрепляет файл, меняет статус, открывает журнал или работает с документами, система опирается на централизованную базу данных. Поэтому данные можно открывать из веб-браузера, мобильного приложения и связанных сервисов без ручной пересылки "последней правильной версии" друг другу.

Что хранится в PostgreSQL

Данные	Для чего нужны
<code>users</code>	Пользователи, роли, должности, активность аккаунтов
<code>auth_sessions</code>	Серверные сессии входа
<code>auth_access_requests</code>	Заявки на активацию доступа
<code>auth_password_recovery_requests</code> и логи	Восстановление доступа и история этих операций
<code>requests</code>	Черновики и отправленные заявки, типы, номера, объекты, статусы, payload формы
<code>request_claim_status_history</code>	История офисных статусов заявки
<code>request_files</code> и <code>request_file_links</code>	Метаданные файлов, связь файлов с заявками

Данные	Для чего нужны
document_storage_folders и document_storage_files	Папки и файлы документохранилища, владельцы, общие/личные зоны
outbound_recipients, outbound_address_book_entries, outbound_logs	Адресаты, записная книжка, журнал отправок
request_events	События по заявкам: кто и когда выполнял действия
dictionaries, approved_dictionary_items, dictionary_candidates, user_recent_entries	Словари, подсказки, недавние значения и модерация
measurement_templates и связанные таблицы	Данные шаблонов замеров
web_push_subscriptions	Браузерные push-подписки пользователей
web_push_notification_rules	Правила, кому и по каким событиям отправлять push
web_push_dispatch_log	Безопасный журнал попыток push-отправки без токенов и содержимого файлов
security_events	Единый журнал событий безопасности: входы, отказы, действия с правами, файлами, заявками и системными ошибками
security_alerts	Тревоги по важным событиям: открыта/закрыта, сколько повторов, когда отправлялся Telegram
security_daily_report_runs	След ежедневного отчета безопасности: дата отчета, период, канал, статус отправки и краткая сводка

Важное уточнение: PostgreSQL хранит структуру, связи, права, статусы, метаданные и часть рабочих данных. Содержимое тяжелых файлов может храниться в серверном файловом хранилище по `relative_path`, а база хранит сведения о файле, владельце, папке, связи с заявкой и размере. Это нормальная архитектура: база отвечает за порядок и связи, файловое хранилище - за тяжелые бинарные данные.

Справочно Что дает PostgreSQL пользователям

Преимущество	Что это дает в работе
Единый источник данных	Не нужно гадать, у кого "последняя версия" заявки
Связи между таблицами	Пользователь, заявка, файлы, статусы и события связаны между собой
Индексы	Журнал, поиск, таймлайн и списки работают быстрее на растущем объеме данных
Уникальные ограничения	Система защищает от дублей там, где они опасны, например по отправленным номерам заявок
Транзакции	Сложное действие выполняется целиком или откатывается, а не остается наполовину записанным
История событий	Можно понимать, кто и когда создал, отправил, удалил или изменил данные
Тревоги безопасности	Важные события не тонут в общем журнале, а поднимаются в отдельный список для администратора
Централизованные права	Проверка роли и владельца идет на сервере, а не "на честном слове" в интерфейсе
Масштабируемость	Система рассчитана на рост числа заявок, пользователей, файлов и статусов
Резервное копирование и обслуживание	Базу можно обслуживать и резервировать как нормальное серверное хранилище, а не как россыпь файлов на ПК

Чем это лучше старой схемы “Word + папки + письма”

Старая схема	Foreman с PostgreSQL foreman-db
Заявка лежит на домашнем ПК	Заявка хранится в общей базе
Файлы разосланы по чатам и почте	Файлы связаны с заявкой и документохранилищем
Непонятно, кто что менял	Есть события, статусы и журналы
Дубли номеров ловятся вручную	Часть дублей контролируется ограничениями базы
Поиск зависит от памяти человека	Поиск идет по структурированным данным
Потеря ПК может означать потерю файла	Рабочая информация находится на сервере

Для пользователя вывод простой: Foreman хранит рабочую информацию не “как получится”, а в нормальной серверной базе PostgreSQL `foreman-db`, где заявки, пользователи, документы и статусы связаны между собой.

Читать всем

0.4 Почему это важно для прорабов и офиса

Работа по старинке	Работа через Foreman
Заявка вручную набирается в Word	Заявка создается в форме
Файл лежит на домашнем ПК	Файл прикреплен к заявке или лежит в документохранилище
Офис ждет письмо	Офис видит заявку в журнале
Непонятно, где задержка	Движение видно в таймлайне
Фото и документы разбросаны	Вложения собраны около заявки
Если отвлекли, можно потерять ход работы	Есть черновики
Нужно помнить, кому что отправлял	Есть журнал и статусы

Справочно

0.5 Как Foreman проверяет файлы

Foreman работает с самыми обычными файлами: заявки Word и Excel, PDF, фотографии с объекта, вложения из писем, документы от монтажников, прорабов, субподрядчиков и смежных организаций. На вид это может быть “просто заявка” или “просто фото”, но для компьютера любой внешний файл - это предмет проверки.

Вирусы не спрашивают, важный у вас документ или нет. Они могут прийти в старом Word-файле, Excel-таблице с макросом, архиве, “счете”, “акте”, “фото”, файле от подрядчика или в пересланной кем-то заявке. Одни вредят тихо: тормозят компьютер, майнят криптовалюту, воруют пароли. Другие действуют грубо: шифруют диск, блокируют доступ к файлам, требуют выкуп, крадут данные браузера и пароли от почты, банков и рабочих сервисов.

Проблема одного зараженного компьютера почти никогда не остается личной проблемой одного человека. Если офисный компьютер потеряет рабочие папки, письма, заявки, шаблоны, архивы или доступы, останавливается не “один файл”, а работа людей вокруг него. Это может закончиться простым, потерей времени, нервами и реальными деньгами.

Foreman использует отдельный антивирусный контур. Проверку выполняет Kaspersky через внешний проверочный API. Касперский - российская компания мирового уровня и одна из самых известных компаний в области кибербезопасности; ее решения используют не для красоты, а чтобы заранее отсеивать известные угрозы и подозрительные файлы.

Что проходит через этот контур:

Откуда пришел файл	Где проверяется
Письмо с вложением на почту чат-бота	Telegram-бот проверяет файл перед сохранением на сервере
Файл, присланный в Telegram	Telegram-бот проверяет файл перед добавлением к заявке или в рабочие файлы
Файл в веб-документохранилище	Backend Foreman проверяет файл перед сохранением в документохранилище
Фото или файл из мобильного приложения	Backend Foreman проверяет файл перед привязкой к заявке

Единый рабочий набор форматов включает документы, таблицы, фотографии, текстовые файлы и строительные чертежи: pdf, doc, docx, xls,xlsx, rtf, txt, jpg, jpeg, png, dwg, dxf. Все они проходят через один и тот же защитный контур. Для пользователя это означает простое правило: рабочий формат сам по себе не является "безопасным по умолчанию", он все равно проверяется и получает понятную отметку.

Foreman смотрит не только на расширение, но и на реальное содержимое файла. Если вредоносную программу, архив, HTML-страницу или другой неподходящий файл просто переименовать в .docx, .pdf, .jpg или другой рабочий формат, система должна увидеть несоответствие и заблокировать такую подделку еще до отправки в Kaspersky. Если файл действительно похож на рабочий документ, фото или чертеж, он проходит дальше не как "уже безопасный", а как файл, допущенный к антивирусной проверке: сначала по отпечатку, а при необходимости - с отправкой самого файла в Kaspersky.

Самый правильный путь - создавать заявки прямо в Foreman Web или в мобильном приложении Foreman. Такая заявка не является внешним Word- или Excel-файлом, не несет внутри макросы, скрытые вложения и чужой исполняемый код. Пользователь вводит данные в форму, а Foreman сохраняет их как структурированную запись в базе: номер, объект, строки, размеры, количество, примечания, статус и события.

Это и есть главное преимущество форм Foreman: сама заявка, созданная в веб-браузере или мобильном приложении, по сути "стерильна" как рабочая запись. Она не может принести вирус как вложенный офисный файл, потому что это не файл с макросами, а данные, введенные через интерфейс. Если к такой заявке прикрепляют фото или документ, тогда уже проверяется именно прикрепленный файл.

Чуть хуже, но все равно разумный вариант - если человек пока делает заявку у себя на ПК в Word или Excel, включать в отправку e-mail чат-бота. Тогда бот сможет проверить письмо с файлами перед тем, как они попадут в общий рабочий поток.

Главная мысль простая: не подставлять коллег небрежной отправкой файлов. Все работают в одной компании, и файл, отправленный "на авось", может ударить не по отправителю, а по офису, бухгалтерии, менеджеру, прорабу или руководителю.

Проверка идет в несколько слоев:

Слой	Что делает	Что это дает
Быстрая локальная проверка	Проверяет расширение, размер и опасные признаки файла	Отсекает очевидно неподходящие или слишком тяжелые файлы
Проверка типа файла	Сравнивает файл с ожидаемым типом, чтобы <code>.docx</code> не оказался исполняемой программой	Защищает от простой маскировки
HTML Smuggling Guard	Не принимает файлы-страницы и браузерные сценарии: <code>html</code> , <code>htm</code> , <code>mhtml</code> , <code>hta</code> , <code>js</code> , <code>vbs</code> , <code>lnk</code> , <code>scr</code> , <code>iso</code> , <code>svg</code> , <code>zip</code> ; дополнительно смотрит, не спрятан ли HTML/JavaScript внутри <code>txt</code> , <code>doc</code> , <code>xls</code> , <code>rtf</code>	Закрывает опасный прием, когда браузер может выполнить файл как код и сам собрать троян на компьютере сотрудника
Проверка через Kaspersky	Сначала отправляет цифровой отпечаток файла, а если по нему нельзя получить уверенный ответ, отправляет сам файл во внешний антивирусный API Касперского	Помогает поймать известные угрозы без установки тяжелого антивируса на тариф Amvera
Журнал безопасности	Записывает безопасную строку о проверке: когда, откуда, имя файла, итог	Позволяет спокойно понять, что произошло, без чтения содержимого файла

Отдельно важно про файлы-страницы. Foreman не принимает `html`, `htm`, `mhtml`, `hta`, `js`, `vbs`, `lnk`, `scr`, `iso`, `svg` и `zip` как обычные рабочие вложения. Причина простая: браузер может не просто показать такой файл, а выполнить его как код. В современных атаках бывает так: человек открывает вроде бы “обычный HTML-отчет”, а браузер внутри себя собирает вредный файл и предлагает скачать или открыть его. Поэтому Foreman не спорит с такими файлами и не просит сотрудника “быть внимательнее”, а просто не пускает их в рабочий поток.

Цифровой отпечаток файла часто называют `хешем`. Это не сам файл и не его текст, а короткий контрольный номер, рассчитанный по содержимому файла. Похоже на отпечаток пальца: по нему можно узнать, встречался ли такой файл раньше и считает ли его антивирусная база опасным.

Почему файл не всегда сразу отправляется целиком? Потому что сначала правильнее спросить у Касперского по отпечатку. Это быстрее, меньше нагружает интернет и внешний сервис, а главное - лишний раз не передает наружу содержимое рабочих документов. Если отпечаток уже известен, ответ можно получить без отправки самого Word, Excel, PDF или фотографии. Если уверенного ответа по отпечатку нет, тогда система переходит к более глубокой проверке и отправляет сам файл.

Foreman не считает неоднозначный ответ безопасным. Зеленая отметка ставится только тогда, когда Kaspersky вернул чистый результат: `Green`, `clean` или смысловой ответ “угрозы не обнаружены”. `Red` - это опасный файл, `Yellow` - подозрительный файл. `Grey`, `not categorized`, `pending`, `no_verdict` или временная ошибка API не показываются как “чисто”: Foreman сохраняет обычный рабочий файл с предупреждающей отметкой и не притворяется, что проверка полностью завершена.

Если Касперский или локальная проверка считают файл опасным или подозрительным, Foreman не делает вид, что все нормально. Файл получает красную или предупреждающую отметку и не проходит как обычный чистый документ. Если внешний сервис временно не дал окончательный ответ, система фиксирует это в журнале отдельным статусом: такой случай не скрывается “под ковром”.

Для писем с опасными вложениями добавлена отдельная поддержка сотрудников. Если прораб отправил или переслал письмо с вирусом чат-боту и одновременно поставил в копию менеджеров или руководителей, Foreman, получив сигнал о вирусе по результатам антивирусной проверки, создает и рассылает ALARM-письмо всем участникам: отправителю, получателям и получателям в копии, кроме самого адреса бота. В письме указывается, кто прислал исходное письмо, когда оно пришло, по какой заявке или заявкам обнаружен вирус, какие файлы не прошли проверку Kaspersky и какое название угрозы передал Касперский, если он его сообщил.

ALARM-письмо отправляется не только при первичном обнаружении вируса. Если сначала Kaspersky дал неоднозначный ответ, например `Grey , not categorized` или "недостаточно данных для вывода", Foreman сохраняет файл с предупреждением и продолжает повторные проверки. Если позже, даже когда исходное письмо уже обработано, Kaspersky окончательно определит файл как опасный или подозрительный, Foreman отправит такое же ALARM-письмо отправителю и участникам исходного письма, кроме адреса самого чат-бота. Это нужно потому, что антивирусные базы постоянно обновляются: по новому вирусу в первую минуту может еще не быть уверенного ответа, а окончательный опасный вердикт может появиться позже.

Такое ALARM-письмо не прикладывает опасный файл повторно. Оно дает короткую рабочую инструкцию: не открывать файл, проверить компьютер офисным антивирусом, если файл уже открывали, сообщить системному администратору и отключить компьютер от сети. Это важно: сотрудник не остается один на один с красной плашкой и не должен гадать, кого предупреждать.

При этом опасный файл не "исчезает бесследно". Он сохраняется как карантинная улика и виден в Foreman рядом с заявкой как внешнее вложение с красной отметкой `вирус`. Обычная кнопка скачивания для такого файла заменяется на `Скачать на свой риск`: открыть его можно только осознанно, понимая, что дальше ответственность уже на человеке и офисном антивирусе.

Важно: журнал безопасности не показывает содержимое файлов, пароли и токены. Он нужен не для любопытства, а для спокойного контроля: файл был проверен, откуда он пришел, какой итог получила система.

Для сотрудников и администратора есть простые команды в Telegram:

Команда	Что показывает
<code>покажи антивирус</code>	Последние проверки файлов: веб, мобильное приложение, письма и Telegram, если события доступны в журнале
<code>покажи проверки файлов</code>	То же самое, более техническое название журнала
<code>покажи заблокированные файлы</code>	Безопасный список файлов, которые система не пустила в обычную работу
<code>покажи безопасность</code>	Админский журнал входов, отказов гостю, лимитов скачивания и скачиваний файлов без паролей, токенов и содержимого документов

В веб-браузере Foreman рядом с внешними вложениями и файлами документохранилища тоже появляется понятная отметка. Если рядом с файлом стоит знак `Kaspersky` и написано `угроз не найдено`, файл прошел проверку и его можно скачивать обычной кнопкой. Если вместо этого показана черная лента `ВИРУС`, предупреждение о подозрительном файле, незавершенной проверке или недостатке данных для вывода, Foreman не прячет документ навсегда, но меняет кнопку на `Скачать на свой риск` и перед скачиванием еще раз предупреждает сотрудника.

Важно: подтверждение риска проверяется не только кнопкой в браузере, но и сервером. Если попытаться скачать рискованный файл из документохранилища прямой ссылкой без подтверждения, сервер откажет. Рискованный файл также нельзя обычным действием прикрепить к заявке или отправить по e-mail из документохранилища: сначала нужен нормальный чистый вердикт Kaspersky. Такой файл нельзя открывать "просто посмотреть": его скачивают только осознанно и только рассчитывая на защиту офисного антивируса.

Чтобы диагностическая страница не была "для красоты", бот регулярно отправляет маленький контрольный PDF в Kaspersky. Поэтому карточка антивируса на странице статуса смотрит не только на наличие токена, а на свежий успешный ответ Касперского. Если свежего ответа давно нет, страница показывает предупреждение.

Для архивных писем и внешних вложений предусмотрена фоновая ретро-проверка. Она не запускается от клика сотрудника по карточке, чтобы не тормозить веб-браузер и не гонять сотни файлов во внешний сервис внезапно. Бот выполняет такую проверку в фоне, записывает итог в журнал и ставит служебную отметку `security_backfill_complete.json`, чтобы не повторять архив заново при каждом запуске. После этого архивные файлы тоже получают понятную отметку в веб-карточке.

Если в списке заблокированных файлов пусто, это хороший нормальный результат. Если там появилась запись, не нужно открывать этот файл вручную. Достаточно передать администратору строку из журнала: откуда файл пришел, имя, время и итог проверки.

Справочно 0.6 Как Foreman читает заявки через распознавание файлов и `office-converter`

Foreman постепенно перестает быть просто полкой для файлов. Он учится читать входящие заявки до того, как сотрудник открыл документ вручную: находит номер заявки, объект, корпус, секцию, строки изделий, размеры, формулы стеклопакетов, количество и подозрительные места. Если в заявке не видно высоту, пропущена формула или количество написано странно, система может заранее подсветить это в служебном блоке.

Слово `OCR` в технических текстах означает оптическое распознавание текста. Проще говоря, компьютер пытается прочитать изображение страницы так, как это делает человек, но только в машинной форме: выделяет слова, цифры и таблицы, а затем передает их Foreman для структурного разбора.

Важный технический принцип: в Foreman OCR сделан не как один проход, а как **каскад**. Для каждого документа включаются этапы по мере необходимости:

1) `page` — базовый проход по странице/изображению; 2) `table` — дополнительный проход по табличным зонам, если они имеют признаки таблицы; 3) `handwritten` — дополнительный проход для

рукописных, эскизных и шумных зон.

По умолчанию каскад работает в режиме **auto**: если сигналы (`tableScore` , `handwrittenScore` , `metadata` из `sourceMetadata`) выше порога, дополнительный проход выполняется автоматически. У каждого режима есть явный тумблер `off|auto|force` , плюс лимит количества вызовов по одному файлу.

Важно: сначала защита, потом распознавание. Если файл не прошел защитный контур и не получил чистый вердикт, документ не идет дальше в рабочий OCR-поток.

Как обрабатываются форматы сейчас:

Формат	Как идет обработка	Статус
<code>docx</code>	Foreman читает структуру Word-документа своим parser-ом, без внешнего распознавания, если документ нормально собран	Реально работает в контролируемом прод-режиме; сложная верстка требует ручной сверки
<code>xlsx</code>	Foreman читает таблицы своим parser-ом	Реализовано и идёт под контролем реальных рабочих шаблонов
<code>doc</code>	Сначала <code>office-converter</code> переводит старый Word-файл в <code>docx</code> , затем Foreman читает его как современный документ	Controlled pilot после live smoke converter-a
<code>xls</code>	Сначала <code>office-converter</code> переводит старую Excel-таблицу в <code>xlsx</code> , затем Foreman читает ее обычным путем	Controlled pilot после live smoke converter-a
<code>jpg</code> , <code>jpeg</code> , <code>png</code>	После проверки безопасности файл идет в Яндекс Vision OCR, затем Foreman parser разбирает результаты	Для фото с объекта используется каскад <code>page</code> + возможные <code>table</code> / <code>handwritten</code> проходы
<code>pdf</code>	Foreman может использовать текстовый слой PDF или распознавать страницы через Яндекс Vision OCR	Полезно для простых PDF; сложные сканы, чертежи и плохие копии требуют осторожности

`office-converter` нужен для старых форматов `.doc` и `.xls` . Эти файлы относятся к устаревшему Office-поток, где возможны зависания и различия в форматах, поэтому конвертер вынесен в отдельный сервис. Он не получает доступ к базе Foreman, не хранит файлы длительно и работает по токену: задача — перевести файл в `docx` / `xlsx` , а дальше все равно разбор выполняет ядро Foreman.

Результат чтения хранится в PostgreSQL. Полная картина остается в `request_content_extractions.canonical_json` , а аналитика раскладывается по таблицам:

Таблица	Что хранит
<code>ocr_extraction_claims</code>	Сводку по заявке: номер, объект, корпус, секция, источник, метод чтения и уверенность
<code>ocr_extraction_items</code>	Строки изделий: размеры, формулы, количество, единицы, площадь, цвет, ламинацию и другие признаки
<code>ocr_extraction_quality_issues</code>	Замечания по строкам: не найдена ширина, высота, формула, количество или найден конфликт

Теперь это уже не просто «текст из картинки». Foreman превращает документ в рабочие данные и добавляет трассу решений: какой режим запущен, почему выбран дополнительный проход или почему он пропущен, какие зоны помечены как неуверенные.

Практическая польза для прораба и офиса:

- если в заявке пропущена высота, количество или формула, в подтверждающем письме появляется понятное замечание;
- если по одной заявке пришло несколько файлов, они объединяются в единый контур данных по заявке;
- если имя файла или тема письма ошиблись, а внутри документа найден корректный номер заявки, используется номер из содержимого;
- если данные конфликтуют, Foreman не делает «тихую» замену, а отмечает конфликт для проверки.

Письма прорабам строятся бережно: замечания по распознаванию добавляются отдельным блоком и группируются по заявке. `items[0].factoryMarking` остается служебным полем в системе, а в письме используется понятный язык: «ширина», «высота», «формула стеклопакета», «количество».

Для админов это не магия и не «черный ящик»: управляемый контур усиления качества. В боевой эксплуатации его включают через feature flags, лимиты, dry-run для рискованных исходящих действий и canary-валидацию на реальных файлах. По умолчанию сырой OCR-текст и сырой ответ провайдера не пишутся в логи без отдельного технического решения.

Справочно 0.7 Как Foreman проверяется перед выпуском в работу

Foreman - не картинка в браузере и не один скрипт. Это связка веба, мобильного приложения, Telegram-бота, базы данных, файлов, писем и рабочих правил. Поэтому перед выпуском изменений в продакшен программа проходит не одну проверку, а несколько разных слоев тестирования.

Главная мысль простая: тесты нужны не “для программистов”. Они нужны, чтобы пользователь не узнал первым, что после обновления сломался вход, журнал, карточка заявки, план-график, письмо или мобильное приложение.

Хорошее обновление должно вести себя так: старые привычные функции продолжают работать как раньше, а рядом аккуратно появляются новые возможности.

Что именно проверяется

Вид проверки	Зачем это пользователю	Что ловит
Smoke-тесты	Быстро отвечают на вопрос “система вообще жива?": сайт открывается, сервер отвечает, базовые двери не заклинило	Поломку запуска, неработающий health/readiness, мертвый web shell, сбой базового smoke бота
Регрессионные тесты	Защищают привычную работу после обновлений: вход, журнал, карточки, заявки, план-график, письма и мобильные сценарии должны вести себя по-старому	Случаи “вчера работало, сегодня перестало”, случайную смену правил, сортировок, прав, статусов, форматов данных и старых сценариев
Contract-тесты	Проверяют “договор” между частями системы: какие поля есть у заявки, карточки, статуса, вложения, ответа API	Поломку связи веба, мобильного приложения, Telegram-бота, backend и базы данных, когда одна часть поменяла формат, а другая этого не поняла
Integration/e2e-тесты	Проверяют несколько частей вместе. E2E - это End-to-End, сквозной путь почти как у живого пользователя: войти, открыть, создать, отправить, увидеть результат	Ошибки на стыке модулей: backend, маршруты, заявки, план-график, письма, файлы, права и статусы

Вид проверки	Зачем это пользователю	Что ловит
UI-тесты	Проверяют реальное поведение в браузере: кнопки нажимаются, карточки открываются, статусы видны, формы не ломаются	Ошибки, которые не видны в "голом" backend-тесте: неактивная кнопка, сломанный overlay, неверная реакция интерфейса
Boundary-тесты	Проверяют границы: ровно допустимое значение и сразу чуть больше, чем можно	Слишком большие файлы и запросы, лимиты получателей письма, крайние номера заявок, опасные границы путей к файлам
Negative-тесты	Проверяют плохие входные данные: неверный файл, пустой ввод, чужой доступ, неправильный адрес, запрещенный путь	Падение сервера вместо понятного отказа, обход прав, прием опасных вложений, мусорные данные в базе
Security/audit-тесты	Проверяют защитный контур: файл должен пройти проверку, браузер не должен выполнить имя файла как код, гость не должен получить запись, а журнал должен записать итог без содержимого файла, паролей и токенов	Попадание опасного файла в обычное хранилище, утечку секретов в диагностике, XSS в карточках файлов, чтение файла вне хранилища, скрытый обход прав
Fault injection-тесты	Искусственно создают сбой: письмо не отправилось, хранилище отказало, metadata-store не записался, токен устарел	Потерю данных при частичном сбое, отсутствие rollback, повтор recovery-кода, "тихое" проглатывание ошибки
Visual regression-тесты	Сравнивают важные экраны с эталоном, чтобы после правки интерфейс не "поехал"	Съехавшие карточки, сломанные кнопки, некрасивые ошибки загрузки, проблемы мобильного размера экрана
Мутационное тестирование	Проверяет уже не программу, а качество самих тестов: в код временно вносят маленькую ошибку и смотрят, поймут ли ее обычные проверки	Слабые тесты "для галочки", слишком общие ожидания, проверки, которые запускаются, но не доказывают правильный результат

Справочно Что такое "мега-тест" перед деплоем

Перед выпуском новой версии на Amvera запускается общая контрольная проверка. Внутри команды ее могут называть "преддеплойный gate" или "мега-тест".

Если говорить простыми словами:

- **деплой** - это выпуск новой версии программы в работу, чтобы ею начали пользоваться сотрудники;
- **pre-deploy gate** - это контрольная проверка перед выпуском, как шлагбаум перед выездом: если проверка не прошла, новую версию не выпускают;
- **красная проверка** - это не страшное сообщение для сотрудника, а сигнал для технического специалиста: один из автоматических тестов нашел проблему, значит выпуск нужно остановить и разобраться;
- **зеленая проверка** - автоматическая проверка прошла успешно.

Он проверяет сразу несколько контуров:

Стадия мега-теста	Что проверяет	Примерный объем
Сборка foreman-api	Приложение должно собраться без ошибок перед выпуском	Это не тесты, а обязательная сборка

Стадия мега-теста	Что проверяет	Примерный объем
Регрессионное ядро	Backend, E2E/сквозные проверки, план-график, real-flow, типы заявок, мобильная совместимость, базовый smoke бота, серверная защита гостевого наблюдателя, базовая проверка proxy-allowlist и контур Security Monitoring	около 1150+ автотестов в текущем составе smoke/regression-контуров
OCR release gate	Сборка backend, OCR parser/worker/services, native document extraction, office-converter client, grouped OCR e-mail blocks, safety gates и выключенный OCR e2e	46 suites / 478 OCR-тестов плюс disabled e2e
Python-тесты Telegram/Foreman request-layer	Внутренний слой заявок, совместимость старых форматов, безопасность владельца, delivery, artifacts, upsert, HTTP routes и proxy-allowlist	115+ автотестов
Базовая быстрая проверка веб-интерфейса (smoke)	/foreman , health, JS/CSS, базовая доступность web shell, порядок журнала, диапазон таймлайна и запрет лишней подгрузки внешних вложений	5 автотестов плюс 3 regression-скрипта
Анализ мобильного приложения	Проверка Dart/Flutter-кода на ошибки анализа	Это статический анализ, не счетчик тестов
Тесты мобильного приложения	Экраны, маршруты, кэш заявок, заявки на заполнения/расходники, защищенное хранение офлайн-черновиков и очереди отправки, проверка безопасного обновления APK, подпись манифеста обновления, корректный запуск Android-установщика, понятное сообщение при отключении доступа, мобильный MFA-вход, постоянный installationId , безопасный deviceProfile , anti-tamper сигналы root/debug/Frida и совместимость с backend	96 автотестов

Итого в обычном мега-тесте проходит примерно 1850+ автопроверок, плюс сборка backend и статический анализ мобильного приложения. Число не высечено в камне: когда добавляются важные сценарии, тестов становится больше. Самым надежным источником точного числа всегда остается свежий вывод `pre_deploy_all.ps1` , `release_check.ps1` , `npm test` и `flutter test` .

Внутри регрессионного ядра проверки распределяются так:

Подстадия regression core	Количество
Jest-тесты foreman-api regression core	117 suites / 1144 tests
Отдельные E2E/smoke-прогоны foreman-api	6 suites / 23 tests
OCR release gate	46 suites / 478 tests
Planning smoke	29
Real-flow smoke	1
Request-types smoke	4
Foreman Mobile compatibility smoke	3
Claim-bot release check smoke	smoke-профиль перед деплоем; полный профиль отдельно гоняет 1857 unittest и 198 pytest

Если хотя бы одна стадия красная, выпуск новой версии запрещен. То есть изменение не должно попасть пользователям просто потому, что “на глаз вроде работает”.

Обычный мега-тест отвечает на вопрос: "Можно ли выпускать новую версию в работу без явной поломки основных рабочих сценариев?"

Мегатест-2 отвечает на другой вопрос: "Достаточно ли проверены рискованные зоны, где ошибка может проявиться не сразу, но дорого обойтись пользователям?"

Он не заменяет обычный мега-тест. Правильный порядок такой:

1. Сначала запускается обычная контрольная проверка перед выпуском (`pre-deploy gate`). 2. Если изменения рискованные, дополнительно запускается мегатест-2. 3. Если любая из этих проверок красная, это значит: автоматическая проверка нашла проблему. Выпуск новой версии запрещен до разбора причины.

В мегатест-2 входят более тяжелые и специальные проверки:

Стадия мега-теста	Что проверяет	Примерный объем
Browser-тесты Foreman Web UI Playwright	Реальные действия в браузере: заявки, e-mail overlay, профиль, push-настройки, безопасное подключение MFA, доверенные устройства, управление браузерными и мобильными профилями, план-график, retry при ошибке расписания и компактное окно событий безопасности	21 браузерный автотест
Visual regression веб-интерфейса Foreman	Внешний вид важных экранов: карточки, кнопки, ошибки и мобильный вид не должны "поехать"	3 визуальных автотеста
Python-тесты fault injection request-layer	Отказоустойчивость: e-mail spool, отключенный пользователь, восстановление пароля, rollback файлов	6 автотестов
Python-тесты boundary/negative request-layer	Опасные границы и плохие входные данные: файлы, лимиты тела запроса, e-mail recipients, path traversal	11 автотестов
Полная регрессия Telegram-бота (<code>full</code>)	Команды, рендер, golden-проверки, старые сценарии, защита от регрессий, реестр запусков, журнал антивирусных проверок, просмотр заблокированных файлов, журнал безопасности Foreman, строгие ответы Kaspersky, ALARM-письма при вирусах, карантинные вложения, запрет браузерных вложений, единая политика рабочих форматов, временные сетевые ошибки Telegram, PostgreSQL-очередь важных писем, защита от повторной SMTP-отправки, диагностика входящих писем, ручной разбор непонятных ответов, push по важным почтовым событиям и админский просмотр почтовых ожиданий	1857 unittest и 198 pytest плюс render/golden-проверки
Backend-тесты антивируса и документохранилища Foreman	Политика <code>clean/no_verdict/api_unavailable</code> , отправка файла в Kaspersky, сохранение рабочих форматов с риск-меткой при неполном ответе, запрет переименованных подделок, быстрая загрузка нормальных файлов с фоновой проверкой, серверный запрет скачивания без принятия риска, запрет прикрепления и e-mail-отправки рискованных файлов, запрет HTML Smuggling-файлов и маскировки HTML/JavaScript внутри рабочих форматов	52 автотеста

Стадия мега-теста	Что проверяет	Примерный объем
Backend-тесты bot-defense/honeypot, MFA и цифрового слежка	Скрытое поле входа не видно человеку, один подозрительный случай только журналируется, временная блокировка включается осторожно, поддельные IP-заголовки не используются как доверенный адрес клиента, MFA не выдает полноценную сессию до кода, известное устройство хранится по hash, расширенный профиль браузера нормализуется, слишком большой профиль не ломает вход, а риск входа пишется в audit без автоматического бана	11 auth-security e2e-тестов плюс целевые unit-тесты нормализации IP, bot-defense, MFA, известных устройств, device profile и risk observations
Backend Security Monitoring MVP	Единый журнал security_events , тревоги security_alerts , ежедневные прогоны security_daily_report_runs , рекурсивная очистка metadata от секретов, правила тревог, дедупликация по alert_key , Telegram mute, ночной вход по Москве, ежедневный отчет за предыдущий московский день и защита от повторной отправки отчета	25 unit-тестов сервиса плюс 9 targeted mutation-мутантов и отдельная UI-проверка окна событий
OCR mutation inventory в стандартном Megatest-2	Быстрая обязательная проверка, что список OCR-мутантов актуален: parser, quantity ambiguity, bundle merge, verification e-mail, office-converter, raw-text leakage, safety gate и Yandex hard flag	test:ocr:mutation:check , 10 targeted OCR-мутантов
Backend security audit Foreman Web	XSS при показе имен файлов, антивирусных меток и полей план-графика, безопасное скачивание attachment , запрет чтения файла вне storage root, security headers веб-страницы, журнал входов/отказов/скачиваний без паролей и токенов, защита источников словарей от прямого API-доступа обычного пользователя, запрет чужому пользователю запускать служебный render/preview/PDF-DOCX и менять sessionMeta чужого черновика, защита общей папки от удаления/переименования/перемещения обычным пользователем, защита замеров от прямого сохранения гостем на уровне сервиса, проверка сессий после отключения пользователя, безопасное отключение доступа сотрудника, сброс пароля и восстановление доступа	30+ автотестов плюс 1 render-regression-скрипт
Backend-тесты гостевого наблюдателя Foreman	Гость не может создать, изменить, отправить, удалить, сгенерировать, прикрепить, загрузить, переименовать, перенести или запустить backfill даже прямым API-запросом; общий гостевой логин держит отдельные сессии посетителей, а скачивание остается только в пределах лимитов	45+ автотестов плюс 1 живая post-deploy проверка на Amvera
Proxy allowlist Foreman через claim-bot	Широкий прокси "любой /api/* " закрыт; проходят только доказанные рабочие маршруты, включая защищенный /api/security/* , неизвестные backend-маршруты режутся на границе claim-bot с 404 foreman_proxy_route_not_allowed , а клиентские X-Forwarded-For , X-Real-IP , CF-Connecting-IP и Forwarded не прокидываются в backend как есть	7 точечных unit-тестов плюс отдельный live-smoke после деплоя
Бережная проверка нагрузки живого сайта	Несколько одновременных безопасных запросов без записи в базу: health/readiness, web shell, журнал, план-график, документохранилище и гостевой просмотр	Последний post-deploy live-прогон: 21 read-only запрос, 4 одновременных потока, p95 259 мс, максимум 323 мс, отказов 0. При MFA у smoke-пользователя user-token часть корректно пропускается

Стадия мега-теста	Что проверяет	Примерный объем
Единый live security smoke после деплоя	Одна команда проверяет живую Amvera после выпуска: health/readiness, публичную диагностическую страницу, запреты гостевого наблюдателя, обычный вход или корректный MFA challenge при наличии актуальных smoke-учетных данных и бережную readonly-нагрузку. Рабочие заявки и файлы при этом не меняются	1 объединяющий smoke-прогон. Последний деплой: endpoints, guest security, auth/readiness и readonly-load OK
PostgreSQL integration на локальной базе	Startup migrations, readiness, login, несовместимая схема, заявка с файлом и журналом, OCR analytics/backfill/report на реальной тестовой базе	2 suites / 8 интеграционных автотестов
Дополнительные backend regression-скрипты	Хранение документов, обновление данных, input/render-логика, внешние вложения и аккуратная подгрузка файлов без лишней нагрузки	3 отдельных regression-скрипта
Foreman Mobile release/security regression	Мобильный контур, который специально не дает незаметно сломать безопасное обновление APK, подпись <code>app-update.json.sig</code> , SHA-256 APK, загрузку APK через Android DownloadManager в стандартную папку «Загрузки», запуск системного Android-установщика, production identity, obfuscation, отсутствие лишних разрешений, постоянный <code>installationId</code> , безопасный <code>deviceProfile</code> , понятную очистку сессии при <code>401</code> и anti-tamper сигналы root/debug/Frida	24+ мобильных автотеста

Полный OCR mutation pilot не гоняется в каждом стандартном Megatest-2, чтобы не делать обычный риск-контур тяжелее без необходимости. Он добавлен в opt-in блок `-IncludeMutationPilot`: там `test:ocr:mutation` реально вносит 10 маленьких OCR-поломок и ожидает, что все они будут пойманы. Последний прогон: `killed=10 survived=0 total=10`.

Итого в `мегатесте-2` в текущем стандартном профиле проходит примерно 2250+ автопроверок, плюс отдельные backend/proxy regression/smoke-скрипты, render/golden-проверки бота и OCR mutation inventory. Это не замена обычного мега-теста, а дополнительный слой для рискованных изменений.

После деплоя у технического исполнителя есть короткая команда для проверки уже живой Amvera: `npm run test:smoke:live-security`. Перед запуском в окружение подставляются рабочие smoke-логин и smoke-пароль, но в руководство, GitHub и журналы они не записываются. Смысл проверки простой: новая версия не просто собралась, а действительно открывается на сервере, пускает обычного пользователя, не дает гостю лишних прав и выдерживает небольшую безопасную нагрузку чтения.

Для изменений в proxy-контуре Foreman дополнительно запускается `scripts\foreman_proxy_allowlist_live_smoke.ps1`. Он проверяет уже живой `claim-bot`: разрешенные рабочие маршруты доходят до защищенного backend-контура, а лишние маршруты вроде `/api/objects`, `/api/critical-mail/status` и `/api/reports/monthly-timeline/preview` не уходят дальше и получают `404 foreman_proxy_route_not_allowed`.

Практическое уточнение по Amvera: после `git push` контейнер может пересобираться и перезапускаться не мгновенно, а несколько минут. Нормальная post-deploy проверка начинается только после стабильного `health/healthz = 200`. Для `claim-bot` в спорных случаях нужно сверять обе production-ветки Amvera, `master` и `main`, чтобы сервис не поднялся со старой ветки.

Практическая оценка устойчивости к ботам на тарифе **Начальный Плюс**

Текущий тариф Amvera: **0.5 CPU** , **1 Гб ОЗУ** , **7 Гб SSD** . Для такого тарифа правильная цель защиты - не обслужить бесконечное число ботов, а быстро начать отвечать лишним запросам **429 Too Many Requests** , сохранив сайт живым для нормальных сотрудников.

Контрольный безопасный live-прогон по живому пользовательскому адресу **<https://claim-bot-tzertis.amvera.io/foreman/>** :

Проверка	Результат
Read-only запросов к живому backend	240
Одновременных потоков	12
p95 времени ответа	1358 мс
Самый медленный ответ	2035 мс
Ошибок	0

Этот прогон не менял заявки и файлы. Он проверял чтение: health/readiness, web shell, журнал, план-график, документохранилище, гостевой просмотр и обычный вход. Это не DDoS-тест и не обещание, что слабый тариф выдержит любую внешнюю атаку. Это рабочая контрольная точка: 12 одновременных read-only потоков живой пользовательский домен выдержал без ошибок. Прямой backend-домен **foreman-api** в тот же день прошел тот же сценарий быстрее: p95 481 мс, максимум 804 мс, ошибок 0.

Действующие лимиты гостевого доступа:

Зона	Лимит	Что это значит простыми словами
Гостевой API после входа	120 запросов в минуту на общего гостя	Все боты под одним гостевым аккаунтом делят один общий лимит
Гостевые сессии	до 100 активных сессий общего гостевого логина	Несколько гостей могут войти одновременно и не выбивать друг друга
Гостевые скачивания	2 одновременных скачивания	Третье параллельное скачивание получает отказ
Гостевые скачивания за окно	20 скачиваний за 10 минут	Массовое выкачивание файлов гостем останавливается лимитом
Повтор одного файла гостем	2 скачивания одного файла за 30 минут на одну гостевую сессию	Обычный повтор возможен, но циклическое скачивание одного файла режется
Общий предохранитель скачиваний	6 одновременных и 80 скачиваний за 10 минут на весь гостевой контур	Много гостевых сессий не должны перегрузить слабый тариф
Предохранитель по IP	3 одновременных и 30 скачиваний за 10 минут с одного нормализованного IP	Один адрес не должен забирать весь гостевой лимит. Клиентские X-Forwarded-For и похожие заголовки не считаются доверенным IP для блокировок
Неверный вход по логину	10 ошибок за 10 минут	Подбор пароля конкретного логина блокируется
Неверный вход по IP	60 ошибок за 5 минут по нормализованному IP	Один адрес не может бесконечно долбить форму входа. Подставленный X-Forwarded-For не должен создавать новый лимитный счетчик

Зона	Лимит	Что это значит простыми словами
Honeypot/bot-defense на входе	Скрытая проверка формы входа. Один подозрительный признак только записывается в журнал	IP-лимит считает количество попыток, а honeypot помогает заметить, что форму отправляет программа

Перевод гостевого API-лимита в примерное число ботов:

Поведение одного бота	Сколько таких ботов заполнят лимит гостя
1 запрос в секунду	около 2 ботов
1 запрос в 5 секунд	около 10 ботов
1 запрос в 10 секунд	около 20 ботов
1 запрос в 30 секунд	около 60 ботов
1 запрос в минуту	около 120 ботов

После заполнения лимита система должна не “героически терпеть”, а отказывать лишним гостевым запросам кодом 429. Это нормальное поведение защиты: боты получают отказ, а сервер не обязан тратить слабый CPU на их обслуживание.

Дополнительно у Foreman включен honeypot на форме входа. Это тихая ловушка для автоматических программ: живой пользователь ее не видит и не заполняет, а простой бот может заполнить скрытое поле или отправить форму слишком механически. Такой случай попадает в журнал bot-defense.

Разница с IP-лимитом простая. IP-лимит отвечает на вопрос: “С одного адреса слишком много действий?” Honeypot отвечает на другой вопрос: “Форма входа была отправлена как человеком или как программой?” Поэтому это не дублирование, а два разных слоя. Один считает количество, другой замечает подозрительный способ поведения.

Важно: один такой случай сам по себе не блокирует сотрудника. Это сделано специально, чтобы не наказать живого человека из-за странного браузера, автозаполнения или сетевой задержки. Временная блокировка применяется осторожно: только когда подозрительное поведение повторяется или складывается с другими тревожными факторами, например быстрыми попытками входа и большим числом неверных паролей. Администратор видит журнал, может снять активную блокировку и при необходимости перевести защиту в режим “только журнал”. Honeypot дополняет rate-limit и MFA, но не заменяет WAF/anti-DDoS и внешний security-мониторинг.

Отдельно проверено доверие к проху-заголовкам. Foreman больше не строит блокировки, rate-limit и нормализованный clientIp по клиентским X-Forwarded-For, X-Real-IP, CF-Connecting-IP или Forwarded. Эти заголовки могут сохраняться только как сырая диагностика, если дошли до backend, но сами по себе не считаются доказательством адреса пользователя. Через публичный путь claim-bot такие клиентские forwarding-заголовки дополнительно отбрасываются на проху-границе. На живой Amvera проверялся запрос входа с X-Forwarded-For: 8.8.8.8 и X-Real-IP: 9.9.9.9: в bot-defense не записался подставленный адрес.

Что можно считать честным выводом:

- внутренний гостевой доступ защищен от раскочки тяжелыми действиями: гостю запрещены создание, изменение, отправка, загрузка, генерация документов, тяжелые preview/render и админские операции;

- массовое гостевое чтение ограничено общим лимитом 120 запросов в минуту;
- массовое скачивание гостем ограничено отдельно;
- IP-лимиты не должны обходиться простой подстановкой `X-Forwarded-For` снаружи;
- живой сайт на слабом тарифе проверенно выдержал 12 одновременных read-only потоков без ошибок;
- агрессивную внешнюю атаку по публичным страницам без входа нельзя честно оценивать как “выдержит N ботов” только по backend-лимитам, потому что такие запросы приходят до гостевой роли. Для них безопасный вывод пока такой: 12 одновременных потоков проверены, более жесткий боевой DDoS на продакшене специально не запускался, чтобы не положить рабочий сайт своими же руками.

Подробное объяснение типов проверок находится выше, в таблице “Что именно проверяется”. Для обычного сотрудника смысл простой: эти проверки нужны не ради отчетности. Они заранее ловят ситуации, когда программа вроде бы открывается, но где-то на краю сценария может потерять файл, неправильно отказать пользователю, сломать связь между телефоном и сайтом или показать кривой экран.

Справочно **Контроль качества защитных тестов**

Кроме обычного мега-теста, у Foreman есть отдельная техническая проверка качества самих тестов. Ее смысл простой: в копии кода специально ломают важное защитное правило и смотрят, заметят ли это автотесты.

Если тесты заметили искусственную поломку, значит защита проверяется не “для галочки”. Если не заметили - такой тест нужно усилить до выпуска.

Эта проверка закрывает самые рискованные зоны:

Зона	Что пытаются сломать	Что должно произойти
Вирусы и опасные файлы	Опасный или непроверенный файл пытаются провести как обычный чистый документ	Тесты должны остановить такую ошибку
OCR и office-converter	OCR пытаются заставить принять неоднозначное количество, потерять конфликт во втором вложении, отключить allowlist verification e-mail, сконвертировать неподдержанный Office-файл, сохранить raw text без флага или обойти clean security gate	Тесты должны поймать искажение заявки, утечку сырого текста и опасный запуск OCR до чистого вердикта
Документохранилище	Файл пытаются скачать или открыть в обход безопасных правил	Тесты должны поймать обход
Гостевой доступ	Гостю пытаются разрешить запись, отправку писем, тяжелые отчеты или обход лимитов	Тесты должны подтвердить, что гость остается только наблюдателем
Права пользователей	Обычному пользователю пытаются дать чужие черновики, админские действия или управление пользователями	Тесты должны отказать
Журнал безопасности	В журнал пытаются записать пароль, токен, cookie или открыть журнал не-администратору	Тесты должны защитить секреты
Security Monitoring	Тревогу пытаются не создать, продублировать лишний раз, отправить Telegram чаще mute-окна или дважды выслать один ежедневный отчет	Тесты должны поймать потерю события, поломку дедупликации и повторную отправку

Зона	Что пытаются сломать	Что должно произойти
Сессии и восстановление доступа	Старый токен, старая сессия или просроченный recovery-код пытаются оставить рабочими	Тесты должны закрыть доступ
Web cookie-auth	Веб-вход пытаются вернуть к хранению основного токена в <code>localStorage</code> или отдать секрет в ответе <code>login</code>	Тесты должны подтвердить, что веб-сессия держится через cookie и не раскрывает основной секрет странице
Доверие к IP	Поддельный <code>X-Forwarded-For</code> или <code>X-Real-IP</code> пытаются использовать как новый адрес для обхода <code>rate-limit</code>	Тесты должны показать, что блокировки строятся по нормализованному IP, а сырые заголовки остаются только диагностикой
Telegram-команды	Служебные команды пытаются открыть неадминистратору или вывести через них секреты	Тесты должны заблокировать это
План-график и веб-страницы	В поля пытаются подставить строку, которая могла бы выполняться в браузере как код	Тесты должны показать ее только как текст
Подтверждающие e-mail прорабам	В письмо пытаются одновременно вставить персональный и гостевой доступ, вернуть отгруженные заявки в мини-таймлайн или снять ограничение <code>max 10</code>	Тесты должны поймать такую ошибку

Хороший итог выглядит так: все искусственные поломки пойманы, ни одна не “выжила”. В техническом отчете это записывается примерно так (актуальный ориентир на 21.06.2026):

```
Mutation pilot summary: killed=40 survived=0 total=40
Backend mutation pilot summary: killed=64 survived=0 total=64
OCR mutation pilot summary: killed=10 survived=0 total=10
```

Для обычного пользователя смысл этой строки такой: проверялась не только программа, но и надежность самих проверок. Foreman должен заранее ловить ошибки в местах, где потеря защиты могла бы привести к вирусному файлу, чужому доступу, утечке секретов или поломке рабочих данных.

Справочно

Насколько программа покрыта тестами

Честно: нельзя одной красивой цифрой сказать “Foreman покрыт на 100%”. Такая фраза была бы обманчивой. Важнее другое: тестами закрываются основные рабочие контуры, где ошибка сразу ударит по пользователю.

Сейчас автоматическими проверками закрываются:

Раздел	Что проверяется
Вход и безопасность	Неверный логин не дает 500, защита от подбора пароля, роли и права, <code>HttpOnly</code> cookie-сессия веба, <code>CSRF</code> -контур и защита <code>rate-limit</code> от поддельных <code>forwarding</code> -заголовков
Журнал заявок	Загрузка, порядок заявок, карточки, старые и новые форматы
Создание заявок	Номера, обязательные поля, черновики, отправка, вложения

Раздел	Что проверяется
План-график	Карточки, даты, перенос, статус <code>Изготовлено</code> , итоги, архивные объекты
Таймлайн и отчеты	Формирование периода, PDF, письма в тестовом режиме
Документохранилище и файлы	Загрузка, скачивание, типы файлов, ограничения вложений, безопасное скачивание через <code>attachment</code> , отказ от путей вне <code>storage root</code>
Подтверждающие письма прорабам	Текст подтверждения, мини-таймлайн последних активных заявок с e-mail отправителя, отсутствие отгруженных заявок, лимит <code>max 10</code> , выбор персонального или гостевого доступа без дублирования
Антивирусный контур файлов	Проверка безопасного сохранения, строгая трактовка ответов Kaspersky, аудит-журнал без секретов, Telegram-просмотр последних проверок и заблокированных файлов, ALARM-письма участникам опасного письма, показ карантинной улики в Foreman, серверный risk-gate документохранилища, XSS-защита меток файлов и рискованных текстовых полей в браузере
OCR, office-converter и structured extraction	Native parser для <code>docx/xlsx</code> , controlled path для <code>doc/xls</code> через <code>office-converter</code> , OCR safety flags, dry-run, grouped OCR remarks in foreman emails, multi-attachment merge, canonical claim number, row-level quality issues, analytics tables <code>ocr_extraction_claims/items/quality_issues</code> , PostgreSQL integration gate и targeted OCR mutation checks
Журнал безопасности Foreman Web	Входы, неудачные входы, отказ гостю в запрещенном действии, превышение гостевого лимита скачивания и скачивание файлов пишутся в отдельный служебный журнал без паролей, токенов, cookie, содержимого файлов и полных путей на диске; читать журнал может только администратор
Security Monitoring и тревоги	События пишутся в PostgreSQL, важные случаи превращаются в тревоги, повторы дедуплицируются, Telegram не засыпает администратора одинаковыми сообщениями, ночной вход считается по <code>Europe/Moscow</code> , ежедневный отчет собирается за предыдущий московский день, а закрытие тревоги оставляет след <code>alert_closed</code>
Гостевой наблюдатель	Просмотр разрешен, запись запрещена: заявки, план-график, документохранилище, файлы, отчеты, тяжелые render-preview и прямые API-запросы проверяются на сервере; после деплоя это дополнительно подтверждается живой проверкой на Amvera
Proxy allowlist <code>claim-bot</code> -> Foreman API	Через публичный <code>claim-bot</code> проходят только заранее разрешенные рабочие API-маршруты; неизвестные или лишние backend-маршруты блокируются до обращения к <code>foreman-api</code> ; клиентские forwarding-заголовки не передаются в backend как доверенные
Мобильное приложение	Анализ кода, тесты экранов и совместимость с backend
Telegram-бот	Ключевые команды, e-mail, вложения, статусные проверки, request-layer, обработчик временных сетевых ошибок Telegram

Что не гоняется автоматически перед каждым обычным деплоем:

- реальные массовые письма наружу;
- реальные Telegram API в боевом чате;
- реальные Google Sheets с живыми изменениями;
- полный визуальный контроль всех экранов;
- ручная проверка на каждом телефоне.

Это сделано специально: тесты не должны случайно отправлять настоящие письма, менять боевые таблицы или шуметь сотрудникам. Такие вещи проверяются отдельными аккуратными сценариями.

Откуда берется оценка тестов по 1000-балльной шкале

Оценка тестового контура, например `875 / 1000`, - это внутренняя риск-шкала Foreman. Она показывает зрелость автоматических проверок и их способность защищать рабочую систему от регрессий.

Оценка строится на фактах, а не на ощущениях:

Блок оценки	Что учитывается
Реальные бизнес-риски	Закрыты ли заявка, Журнал, План-график, письма, файлы, push, роли, мобильное приложение
Регрессии и мутации	Есть ли тесты, которые ловят уже известные опасные поломки, и проходят ли targeted mutation checks, включая отдельный OCR mutation inventory
Интеграционные проверки	Проверяются ли не только отдельные функции, но и связка backend, web, mobile, bot, база, файлы, письма, OCR analytics и converter
Безопасность и роли	Проверяются ли гость, прораб, менеджер, администратор, восстановление доступа, SSRF, файлы и download-guards
Повторяемость	Можно ли запустить проверки командой, а не вспоминать руками
Место в процессе	Встроены ли проверки в преддеплойный порядок: основной мега-тест и мегатест-2 перед отправкой на Amvera
Документация	Понятно ли, что запускать и когда деплой запрещен

Число `875 / 1000` означает высокий уровень зрелости тестового контура Foreman для текущего масштаба проекта. Оно держится на усилении Security Monitoring и новом OCR-контуре: появились проверки правил тревог, дедупликации, Telegram mute, московского периода ежедневного отчета, защиты от повторной отправки, компактного окна событий в UI, OCR parser/boundary cases, PostgreSQL analytics gate и targeted OCR mutation checks. При этом оценка остается честной: внешние зависимости - Amvera, Gmail/SMTP, Telegram API, Google-доступы, live Yandex OCR, live office-converter и установка APK на настоящий Android - проверяются отдельным controlled smoke/runbook, а не подменяются локальной имитацией.

Эта шкала не заменяет внешний пентест или официальный аудит. Ее практическое назначение - показать, какие защиты уже подтверждены тестами, а какие зоны требуют отдельной live-проверки или внешней оценки.

Справочно

Почему одного ручного клика недостаточно

Ручная проверка полезна, но человек обычно проверяет один путь: открыл сайт, вошел, увидел журнал. Автотесты проходят больше вариантов:

- неправильный логин;
- граничные номера заявок;
- пустые и старые данные;
- права администратора, прораба, менеджера и гостя;
- сохранение карточки после обновления страницы;
- пересчет итогов в план-графике;
- письмо без реальной массовой отправки;
- мобильную совместимость;
- ситуации, где раньше уже были ошибки.

Тесты сильно снижают риск, но не обещают магию. Они не заменяют здравый смысл, резервные копии, контроль базы данных и аккуратную эксплуатацию. Если изменилась платформа Amvera, закончилась база, поменялись пароли почты или Google-доступы, это уже не обычная ошибка кнопки в интерфейсе, а эксплуатационная проблема.

Но для рабочих изменений в Foreman правило такое: перед продакшеном программа проходит автоматический контроль, а важные баги закрепляются тестами, чтобы не возвращаться снова.

Читать всем

Перед началом: пользователи, вход и права

Читать всем

В.1 Вход в веб-браузере

Основной адрес: <https://claim-bot-tzertis.amvera.io/foreman/>

Прямой backend-адрес для диагностики: <https://foreman-api-tzertis.amvera.io/foreman/>

В обычной работе сотруднику лучше использовать основной адрес на домене `claim-bot`. Он совпадает с тем адресом, который Telegram-бот показывает в ссылках, и проходит через штатный шлюз Foreman. Прямой домен `foreman-api` нужен техническому специалисту для проверки backend и может использоваться как резервный вход, если нужно понять, где именно проблема: на шлюзе `claim-bot` или на backend.

Диагностическая страница для спокойной проверки состояния системы:

<https://claim-bot-tzertis.amvera.io/status>

Эта страница сделана не для обычной работы с заявками, а для проверки "система жива или нет". Она показывает понятный статус сайта, базы данных, входа пользователей, файлов, антивирусной проверки файлов, почты, Telegram-бота, Google Sheets, фоновых задач и интеграций. Страница скрывает секреты и не показывает пароли.

Если сотрудник боится, что "что-то сломается, а никто не узнает", важно объяснить: кроме обычного интерфейса есть отдельная диагностическая страница. Если там зеленый статус, система в целом готова к работе. Если там красный или желтый статус, это повод не паниковать, а передать администратору код диагностики.

Порядок:

1. Открыть ссылку в браузере. 2. Ввести `логин`. 3. Ввести `пароль`. 4. Нажать вход. 5. После входа вверху появится имя пользователя и должность.

Если пароль не виден, используется кнопка показа/скрытия пароля.

Читать всем

В.2 Вход в мобильном приложении

Порядок:

1. Открыть приложение Foreman. 2. Ввести тот же логин и пароль, что для веба. 3. Нажать вход. 4. После входа откроется главный экран.

Мобильное приложение работает через тот же сервер, что и веб.

V.3 Сколько держится вход

Сейчас серверная сессия действует около 30 дней .

Повторный вход потребуется, если:

- пользователь нажал Выйти ;
- прошло около 30 дней;
- администратор поменял пароль;
- администратор отключил пользователя;
- браузер очистил данные сайта;
- приложение переустановили;
- на телефоне очистили данные приложения;
- старый токен был получен до продления срока сессии.

Важно: если человек вошел до изменения срока сессии, старый вход мог жить меньше. После одного нового входа должен применяться новый срок.

V.4 Роль и должность

В Foreman есть два разных понятия:

Понятие	Что означает
Должность	Как человека уважительно показывать в интерфейсе: прораб, зам. директора, менеджер
Роль	Какие действия разрешены в системе: admin, foreman, manager, guest

Пример: человек может быть зам. директора , но технически иметь роль, которая разрешает просмотр и работу с заявками.

V.5 Основные роли

Роль	Что может
Администратор	Управлять пользователями, видеть расширенные разделы, менять права
Сотрудник / прораб / менеджер	Работать с заявками, черновиками, вложениями, своими документами
Наблюдатель	Смотреть журнал, карточки, таймлайн, скачивать доступные файлы, читать общие документы

V.6 Гостевой наблюдатель

Гостевой доступ нужен для человека, которому можно смотреть, но нельзя менять.

Текущий гостевой аккаунт:

Поле	Значение
Логин	vrem.prov
Пароль	vrem.prov
Должность	наблюдатель

Общий гостевой логин не означает одну общую сессию. Каждый вошедший гость получает свою серверную сессию посетителя. Поэтому если один подрядчик вошел и не нажал **Выйти**, другой гость не должен быть выбит из системы только из-за этого. По умолчанию гостевой логин может держать до 100 активных сессий, а обычные пользователи - до 3.

Наблюдателю разрешено:

- открыть **Журнал** ;
- открыть карточку заявки;
- скачать доступные вложения из карточки;
- открыть **Таймлайн** ;
- двигать ползунки таймлайна;
- нажать **Обновить** ;
- нажать **Скачать PDF** , если скачивание не превышает лимит;
- открыть **Документохранилище** ;
- видеть только **Общие документы** ;
- переключать **Строки** / **Превью** ;
- сортировать по имени, типу, дате;
- скачивать файлы из общих документов в пределах лимита.

Наблюдателю запрещено:

- создавать заявку;
- редактировать заявку;
- удалять заявку;
- загружать файлы;
- менять план-график;
- добавлять и редактировать примечания в план-графике;
- менять статусы, даты, объекты и права план-графика;
- формировать новые PDF/DOCX-документы по заявкам;
- создавать папки;
- переименовывать папки;
- удалять папки;
- переименовывать, переносить и копировать файлы;
- прикреплять файлы к заявкам;
- отправлять файлы по e-mail;
- запускать служебные проверки и backfill.

Отдельно включены лимиты, чтобы гостевой доступ нельзя было использовать для перегрузки слабого тарифа Amvera. По умолчанию одна гостевая сессия может держать не больше 2 одновременных скачиваний и не больше 20 скачиваний за 10 минут. Повтор одного и того же файла ограничен 2 скачиваниями за 30 минут на сессию. Дополнительно есть общий предохранитель всего гостевого контура и отдельный предохранитель по IP. Обычных сотрудников эти гостевые лимиты не касаются.

Важно: это не только скрытые кнопки в браузере. Сервер тоже проверяет роль `guest`. Даже если кто-то вручную вызовет API-запрос или искусственно вернет кнопку на страницу, Foreman должен отказать до изменения данных.

Справочно

В.7 Безопасность Foreman: общие правила

Этот блок относится сразу ко всем трем направлениям: веб-браузеру, мобильному приложению и Telegram-боту. Дальше по разделам правила не дублируются, чтобы руководство не превращалось в кашу из повторов.

Главный принцип: интерфейс может скрывать кнопки для удобства, но реальная защита должна проверяться на сервере. Если у пользователя нет права, сервер должен отказать даже тогда, когда кнопку случайно удалось увидеть или вызвать напрямую.

Мера	Что защищает	Как проявляется для пользователя
Авторизация по логину и паролю	Вход в систему	Без входа закрытые API-действия недоступны
Защита от подбора пароля	Вход пользователей	После серии неверных попыток вход временно ограничивается сообщением Слишком много неверных попыток входа. Попробуйте позже.
HttpOnly cookie-сессия в веб-браузере	Защита веб-входа от кражи токена обычным JavaScript	Foreman Web остается в сессии после входа, но страница не читает основной секрет авторизации из <code>localStorage</code>
Bearer-токен для API-клиентов	Мобильное приложение, Telegram-бот и служебные API-запросы	При 401 сессия сбрасывается, пользователь возвращается ко входу
Проверка роли на сервере	Защита от чужих действий	Гость и обычный пользователь не могут делать админские операции
Нормализация IP на сервере	Rate-limit, bot-defense и журналы безопасности	Подставленные <code>X-Forwarded-For</code> и похожие заголовки не считаются доверенным IP для блокировок
Проверка автора заявки	Защита чужих черновиков и вложений	Чужую заявку можно смотреть, но нельзя править
Блокировка действий по статусу	Защита уже отправленных заявок	У отправленной заявки нельзя свободно менять вложения
Разделение личных и общих документов	Защита файлов	Личные документы видит владелец, общие доступны по правилам общего раздела
Ограничение гостя	Безопасный просмотр для внешнего человека	Гость смотрит и скачивает, но не меняет данные
Ограничения на загрузку файлов	Защита от тяжелых и случайных загрузок	В документохране загрузка ограничена количеством и размером

Мера	Что защищает	Как проявляется для пользователя
Антивирусная проверка файлов	Защита от опасных вложений	Рабочие файлы проходят проверку перед сохранением; заблокированные файлы не попадают в обычную работу
Honeypot/bot-defense	Защита формы входа от простых автоматических атак	Скрытая проверка помогает заметить автоматическую отправку формы. Один случай только журналируется; ограничение входа возможно только при повторном или явно подозрительном поведении
Security Monitoring	Обнаружение важных событий и тревог	Администратор видит Тревоги, События и Отчёты; критичные события уходят в Telegram и попадают в ежедневную сводку
Выход из аккаунта	Завершение доступа на устройстве	Выйти удаляет локальную сессию

Управленческая оценка безопасности по 1000-балльной шкале

Эта оценка нужна не для красивой цифры, а для разговора с руководством простым языком: где Foreman находится относительно обычных сайтов, отраслевых систем и крупных технологических платформ. Это не сертификат, не гарантия “невзламываемости” и не замена внешнего пентеста. Это осторожная управленческая шкала, собранная по фактическим защитным слоям Foreman и по открытым ориентирам рынка: OWASP Top 10 для веб-приложений, где актуальная ветка уже 2025, OWASP Mobile Top 10/MASVS для мобильных приложений, каталог CISA KEV по реально эксплуатируемым уязвимостям и отчет Verizon DBIR по типовым причинам взломов.

Важно сказать прямо: баллы ниже не являются точным инженерным расчетом “до единицы” и не получены автоматической сертификационной формулой. Это экспертная ориентировочная оценка по реализованным мерам, тестам, журналированию, ограничениям доступа и по тем зрелым практикам, которых у Foreman пока нет. Поэтому диапазоны даны широкими и консервативными, без попытки выдать внутренний проект за зрелую корпоративную SaaS/ECM-платформу.

Важно: оценка ниже намеренно дана консервативно. Foreman уже не является “самописным сайтом с одним паролем”, но и не должен подаваться как промышленная корпоративная платформа. Появился собственный Security Monitoring MVP: события, тревоги, Telegram-оповещения и ежедневный отчет. Но пока не проведены независимый пентест, внешний аудит мобильного приложения, формальная проверка инфраструктуры и подключение зрелого внешнего SOC/SIEM-процесса, завышать уровень было бы неправильно.

Отдельно про SOC и SOMM: Foreman не является корпоративным SOC. По зрелости Security Operations организационный уровень сейчас честно находится примерно на 0.8-1.0 из 5: нет дежурной смены мониторинга, SLA реагирования, формальных playbook, внешнего SIEM и централизованного сбора серверных и сетевых логов. Но прикладной уровень мониторинга именно внутри Foreman уже заметно выше - примерно 1.7-2.0 из 5: сайт не молчит, фиксирует важные действия, умеет тревожить администратора и дает журнал для разбирательства. Это еще не банковский центр мониторинга, но уже и не старый сайт уровня “узнали через неделю, что кто-то удалил страницы”.

Важность отдельных мер берется не из “домашней модели” и не из личной симпатии к конкретной функции. Она определяется по общепринятой инженерной практике и открытым ориентирам безопасности. Например, OWASP рекомендует защищать сессии и токены; хранение основного веб-токена в `localStorage` считается более рискованным при XSS; `httpOnly` cookie снижает доступ JavaScript к сессионному секрету. Но такая cookie сама по себе не делает сайт безопасным: рядом нужны `Secure`, `SameSite`, CSRF-защита, серверные роли, MFA, audit, rate-limit и тесты. Поэтому отдельной мере не приписывается “официальная цена” вроде +50 или +10; оценивается ее вклад в общий защитный слой.

Как читать шкалу:

Диапазон	Что это значит
0-300	Слабая защита: вход, файлы и права держатся в основном на доверии к пользователю
300-550	Базовая защита: есть логин и пароль, но мало серверных проверок, ограничений доступа, журналов и тестов
550-700	Нормальный малый/средний рабочий сервис: основные права и типовые ошибки закрыты, но мало глубокой проверки
700-820	Усиленный рабочий сервис: защита многослойная, есть реальные регрессии, ограничения, журналы и контроль опасных файлов, но еще нет полного набора зрелых корпоративных процессов
820-930	Зрелая отраслевая или корпоративная SaaS/ECM-система: дополнительно есть внешний пентест, WAF/anti-DDoS, SSO, зрелый мониторинг, формальная политика релизов и реагирования
930-990	Уровень крупнейших технологических платформ: большие security-команды, bug bounty, постоянный SOC, независимые аудиты, формальные сертификации и огромный бюджет защиты

Сравнение с рынком:

Категория	Примерный уровень	Почему так
Заброшенный сайт, старая CMS, один пароль на всех	150-350	Часто нет обновлений, журналов, разделения прав и нормальной проверки файлов
Обычный небольшой сайт компании	350-550	Есть форма входа и HTTPS, но мало серверных проверок, ограничений доступа, тестов и контроля файлов
Нормальный внутренний сервис малого бизнеса	550-700	Роли, авторизация и база данных обычно есть, но защита часто не проверяется глубоко

Категория	Примерный уровень	Почему так
Foreman сейчас	760-805	Веб, мобильное приложение и Telegram-бот имеют несколько рабочих слоев защиты: серверные роли, гостевые ограничения, проху-allowlist, антивирусный контур, контроль опасных форматов, журналы безопасности, осторожный honeypot/bot-defense на входе, HttpOnly cookie-сессия для веба, защиту rate-limit от простой подмены forwarding-заголовков, TOTP/MFA для веб-входа и мобильного админского входа, расширенный цифровой слепок браузера, доверенные устройства и наблюдение риск-сигналов входа, Security Monitoring с событиями, тревогами, дедупликацией, Telegram-уведомлениями и ежедневным отчетом, управляемые Web Push-оповещения, мобильную APK-сборку с obfuscation, проверку подписи манифеста обновления и SHA-256 APK, корректный Android install-flow обновления, anti-tamper сигналы root/debug/Frida, управление доверенными мобильными устройствами, безопасное отключение доступа сотрудника с отзывом web/mobile-сессий и блокировкой мобильных устройств, регрессии, мегатест и мегатест-2. Диапазон поднят умеренно: новый monitoring сокращает время обнаружения инцидента, но не заменяет внешний аудит, WAF/anti-DDoS, SOC/SIEM и формальную карту реагирования
Зрелая строительная SaaS/ECM-система	820-930	Обычно добавлены отдельные security-процессы, корпоративный мониторинг, внешние проверки, независимый пентест, корпоративный SSO, централизованное управление MFA, договорные SLA, регулярные аудиты, формальная карта реагирования на инциденты и ответственная эксплуатация безопасности
Крупные платформы уровня Microsoft/Amazon/Яндекс	930-990	Огромные команды безопасности, круглосуточный SOC/мониторинг, WAF/anti-DDoS, независимые аудиты, формальные security-процессы, SSO, bug bounty, сертификации и большой бюджет на защиту

Оценка Foreman по направлениям:

Направление	Оценка	Что уже усиливает защиту	Что еще может поднять уровень
Foreman Web в браузере	775-825	Авторизация, серверные роли, защита гостя, лимиты скачивания, журнал безопасности, проху-allowlist, honeypot/bot-defense на входе, HttpOnly cookie-сессия вместо хранения основного веб-токена в localStorage, нормализация IP для rate-limit, TOTP/MFA с QR-кодом и резервными кодами, выборочная админская галочка Требовать MFA, предупреждение перед подключением MFA, расширенный цифровой слепок браузера, известные устройства по hash, админский просмотр и управление доверенными устройствами, audit-наблюдения риска входа, Security Monitoring в админке с Тревогами, Событиями и Отчётами, Telegram-уведомления по важным тревогам, явное отключение доступа сотрудника с отзывом сессий, отключение наружного X-Powered-By: Express, антивирусный контур, запрет опасных вложений, профильные Web Push-подписки, админские правила push-событий, UI/visual-регрессии	Ужесточить CSP без unsafe-inline, добавить внешний WAF/anti-DDoS, SSO, включить risk-based step-up после периода наблюдения, оформить карту реагирования на инциденты и провести независимый пентест
Telegram-бот и claim-bot	725-780	Проверка ролей, безопасные команды, quarantine/risk-gate файлов, ALARM-письма при вирусах, push по важным почтовым событиям, проху-allowlist, пропуск /api/security/* только как защищенного backend-маршрута, отбрасывание клиентских forwarding-заголовков перед backend, Telegram Bot API для важных тревог и ежедневного отчета безопасности, диагностика, тесты полного regression-контюра	Добавить внешний мониторинг доступности, формальную карту реагирования на инциденты, регулярный внешний аудит и централизованный SOC/SIEM-контур

Направление	Оценка	Что уже усиливает защиту	Что еще может поднять уровень
Foreman Mobile	780-815	Актуальная APK-сборка в production работает только с утвержденным сервером Foreman, запрещает cleartext HTTP, очищает неподходящий сохраненный API-адрес, не использует дефолтный сервисный PIN, выпускается с постоянной production-подписью, нормальным applicationId ru.tzertis.foreman, отключенным backup, собирается с obfuscation, проверяет подпись app-update.json.sig перед доверием к манифесту обновления и затем сверяет SHA-256 скачиваемого APK. APK скачивается через Android DownloadManager прямо в стандартную папку «Загрузки»; Foreman проверяет SHA-256 именно этого файла и только после проверки передает его штатному Android-установщику, а при отсутствии разрешения открывает системную настройку установки из этого источника. Офлайн-черновики и очередь отправки хранятся в защищенном хранилище с миграцией старых данных из SharedPreferences, release-сборка запрашивает только необходимые для работы разрешения, мобильный админский вход поддерживает MFA challenge. Foreman Mobile создает постоянный installationId в защищенном хранилище, отправляет безопасный deviceProfile, backend хранит только SHA-256 hash экземпляра приложения, привязывает мобильные сессии к экземпляру Foreman Mobile на конкретном смартфоне и позволяет администратору заблокировать доступ к мобильному приложению на потерянном смартфоне сотрудника, без блокировки остальных устройств. При отключении сотрудника мобильная сессия получает понятный отказ и очищается локально. Anti-tamper v1 добавляет проверку явных признаков debugger/TracerPid/Frida и audit-сигналов root без сканирования всех приложений на телефоне	Провести независимый мобильный пентест, рассмотреть certificate pinning или Play Integrity, регулярно проверять Flutter-зависимости, протестировать установку/обновление на нескольких реальных Android-устройствах и отдельно проверить поведение при старой версии Android
Система Foreman в целом	760-805	Защита не держится на одной кнопке: права проверяются сервером, файлы проходят антивирусный контур, лишние проху-маршруты режутся до backend, веб-вход переведен на HttpOnly cookie, вход дополнительно прикрыт honeypot/bot-defense и TOTP/MFA, мобильный админский вход также проходит через MFA challenge, мобильная сборка obfuscated, манифест обновления подписан, APK проверяется по SHA-256 и открывается через штатный Android install-flow, rate-limit не доверяет подставленным forwarding-заголовкам, расширенный цифровой слепок браузера, доверенные устройства и риск входа пишутся в audit, мобильные сессии привязаны к hash экземпляра Foreman Mobile, доступ мобильного приложения на потерянном смартфоне можно заблокировать отдельно от остальных устройств, а доступ уволенного сотрудника отключается отдельным серверным действием с отзывом web/mobile-сессий и блокировкой мобильных устройств. Anti-tamper v1 блокирует явные debug/Frida-сценарии мобильного входа и пишет риск-сигналы, важные события могут дублироваться управляемым Web Push, а Security Monitoring создает тревоги, дедуплицирует повторы, не шлет Telegram чаще заданного mute-окна и каждый день отправляет сводку за предыдущие московские сутки. Изменения проверяются тестовыми контурами. Верхняя граница сознательно удержана ниже уровня зрелой корпоративной платформы до внешнего аудита, WAF/anti-DDoS, SSO и внешнего SOC/SIEM	Внешний пентест, WAF/anti-DDoS, SSO, включение risk-based step-up после накопления наблюдений, формальная карта реагирования на инциденты, регулярная проверка зависимостей и секретов

Что уже реально усиливает Foreman для текущего масштаба:

- гость не просто “не видит кнопки”, а ограничен сервером;
- скачивание гостя ограничено по сессии, повтору файла, общему контуру и IP;

- широкий проху "любой `/api/*`" закрыт, оставлены только проверенные маршруты;
- Foreman Web использует HttpOnly cookie-сессию: основной веб-секрет не хранится в `localStorage` и не читается обычным JavaScript страницы;
- Foreman Web поддерживает TOTP/MFA: пользователь подключает приложение-аутентификатор, вводит одноразовый код, получает резервные коды, а администратор может выборочно требовать MFA для нужных сотрудников;
- перед подключением MFA сотрудник видит предупреждение: QR-код создается только после подтверждения, что приложение-аутентификатор уже установлено, а отмена не блокирует вход;
- Foreman видит знакомые веб-устройства по серверному hash и расширенному цифровому слепку браузера: язык, часовой пояс, параметры экрана, платформу, touch-признаки и другие легкие технические признаки; эти данные не дают доступ сами по себе, а помогают аккуратнее оценивать вход;
- администратор в `Моих реквизитах` может открыть таблицу цифровых слепков, выбрать пользователя, посмотреть знакомые браузеры, дать устройству понятное название, отметить доверенное устройство или отозвать доверие;
- risk observations записывают факторы входа в audit-режиме: новое устройство, отличие цифрового слепка, скорость отправки формы, неудачные попытки перед успешным входом, bot-defense и honeypot-сигналы. Пока это наблюдение, а не автоматическая блокировка по "цифровому слепку";
- rate-limit, bot-defense и security events не доверяют клиентским `X-Forwarded-For`, `X-Real-IP`, `CF-Connecting-IP` и `Forwarded` как источнику IP для блокировок;
- Security Monitoring записывает события в PostgreSQL, санитизирует metadata от паролей, токенов, cookie, authorization/session/jwt/csrf и ведет отдельные тревоги для ночного входа, запрета обычному пользователю к админскому API, повторных неудачных входов, всплеска 5xx, смены ролей, MFA, прав план-графика, подозрительных файлов, массового удаления и критичных изменений доступа;
- тревоги не плодятся бесконечно: повтор с тем же `alert_key` обновляет счетчик и время последнего появления, а Telegram не отправляется чаще mute-окна - 30 минут для `critical` и 60 минут для `warning`;
- Telegram Bot API теперь используется не только для рабочих команд, но и как быстрый канал безопасности: критичные тревоги и важные предупреждения уходят администратору в личный Telegram, а ежедневный отчет отправляется за предыдущий московский календарный день;
- в админке Foreman появился раздел `Безопасность`: вкладки `Тревоги`, `События`, `Отчёты`, закрытие тревоги с комментарием и компактное окно событий с внутренней прокруткой, чтобы журнал не превращал страницу в бесконечную простыню;
- живой веб-ответ больше не раскрывает наружу служебный заголовок `X-Powered-By: Express`;
- опасные и маскированные файлы не проходят как обычные рабочие документы;
- Kaspersky-контур и risk-gate не считают неопределенный ответ безопасным;
- безопасность закреплена тестами, мегатестом, мегатестом-2 и live-smoke после деплоя; отдельный браузерный тест проверяет, что случайный клик `Подключить MFA` не создает QR-код и не пугает сотрудника неожиданной настройкой;
- Web Push добавлен как дополнительный управляемый канал коротких оповещений, но не как замена e-mail, Telegram или журналу событий;
- honeypot/bot-defense добавлен как осторожная защита входа от простых ботов: он замечает автоматическую отправку формы входа, записывает такие случаи в журнал и может временно ограничить доступ только при повторном или явно подозрительном поведении;

- мобильная APK-сборка исключает увод пользователя на чужой сервер в production, имеет постоянную production-подпись, нормальный applicationId, собирается с obfuscation и проверяет SHA-256 скачиваемого обновления;
- мобильное обновление теперь защищено подписью манифеста: приложение проверяет `app-update.json.sig` перед тем, как доверять списку версии, ссылке на APK и контрольной сумме;
- мобильное обновление не бросает сотрудника искать APK в памяти телефона: Foreman скачивает APK через Android DownloadManager прямо в стандартную папку «Загрузки», проверяет SHA-256 именно этого скачанного файла, открывает системный Android-установщик и при необходимости переводит пользователя в настройку разрешения установки именно для Foreman;
- Foreman Mobile создает постоянный `installationId` конкретной установки приложения, отправляет безопасный профиль телефона, а backend хранит только hash экземпляра приложения и может отозвать сессии Foreman Mobile именно на потерянном смартфоне, не блокируя сотрудника целиком;
- Foreman Mobile получил anti-tamper v1: явные debugger/TracerPid/Frida-сигналы блокируют мобильный вход и отзывают текущую мобильную сессию, а root-признаки пока работают мягко - как предупреждение и audit/risk-сигнал без автоматической расправы над обычным пользователем;
- Foreman Mobile хранит офлайн-черновики и очередь отправки в защищенном хранилище, автоматически переносит туда старые локальные данные из `SharedPreferences`, а release-сборка запрашивает только необходимые для работы разрешения;
- для увольнения, потери доверия или утечки логина/пароля добавлено отдельное действие `Отключить доступ сотрудника`: оно деактивирует учетную запись, отзывает активные web/mobile-сессии, переводит известные устройства и мобильные профили в заблокированное состояние и оставляет audit-событие. Это не удаление истории и не «очистка следов», а аккуратный стоп-кран доступа;
- веб-версия частично адаптирована для работы со смартфона: план-график, таймлайн и журнал стали компактнее и удобнее для просмотра, хотя это еще не полноценная отдельная мобильная веб-версия.

Источники для ориентиров:

- OWASP Top 10 Web Application Security Risks: <https://owasp.org/www-project-top-ten/>
- OWASP Mobile Top 10 2024: <https://owasp.org/www-project-mobile-top-10/>
- OWASP MASVS: <https://mas.owasp.org/MASVS/>
- CISA Known Exploited Vulnerabilities Catalog: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- Verizon Data Breach Investigations Report: <https://www.verizon.com/business/resources/reports/dbir/>

Читать всем

В.8 Безопасность в веб-браузере Foreman Web

Сессия и токен

Веб-версия Foreman после входа использует серверную cookie-сессию. Основной секрет веб-авторизации хранится в cookie с флагами `HttpOnly`, `Secure` и `SameSite`, а не в `localStorage`.

Что это дает:

- обычный JavaScript страницы не может прочитать основной веб-токен и отправить его злоумышленнику как строку;
- cookie передается браузером автоматически только в запросах к Foreman;
- `Secure` требует HTTPS-соединение;
- `SameSite` снижает риск чужого сайта незаметно выполнить действие от имени пользователя;
- для веб-запросов с изменением данных используется дополнительная CSRF-защита.

Честно: cookie-сессия не делает XSS “безопасным”. Если на странице появится вредный скрипт, он все еще может пытаться действовать внутри открытой вкладки от имени пользователя. Но украсть сам основной веб-токен из `localStorage` стало заметно сложнее, потому что его там больше нет.

Bearer-токены сохраняются для других клиентов Foreman: мобильного приложения, Telegram-бота и служебных API-сценариев. Это нужно для совместимости экосистемы и не означает, что веб-браузер снова хранит основной секрет в `localStorage`.

Что это значит практически:

- на личном ПК можно не входить каждый день;
- на чужом или общем ПК обязательно нажимать `Выйти`;
- если браузер очистил данные сайта, придется войти заново;
- если сервер ответил `401`, веб сбрасывает сессию и просит войти снова.

Читать всем

Двухфакторная защита входа MFA/TOTP

Foreman Web поддерживает двухфакторную защиту входа. В обычной ситуации сотрудник вводит логин и пароль. Если для его учетной записи включена MFA, после правильного пароля Foreman попросит еще один код из приложения-аутентификатора.

Здесь используется TOTP - одноразовый 6-значный код, который меняется примерно каждые 30 секунд. Подойдут Яндекс Ключ, Google Authenticator, Microsoft Authenticator, Aegis, 2FAS и другие приложения, умеющие показывать такие коды. Foreman не привязан к одному конкретному поставщику.

Чем TOTP лучше SMS-кода:

- код работает даже там, где плохо приходят SMS, если приложение уже настроено;
- не нужно платить за SMS и зависеть от оператора связи;
- меньше риск SIM-swap, когда злоумышленник пытается перевыпустить SIM-карту на себя;
- меньше риск задержки, потери или перехвата SMS;
- код создается на телефоне сотрудника и не требует отдельной доставки через сеть оператора.

Честно: TOTP тоже не волшебная броня. Если сотрудник сам покажет код постороннему человеку или потеряет телефон без блокировки экрана, риск остается. Поэтому Foreman дополнительно выдает резервные коды, а администратор может сбросить MFA при потере доступа.

Как подключение сделано безопасно для обычного сотрудника:

1. В Мои реквизиты сотрудник нажимает Подключить MFA . 2. Foreman сначала показывает предупреждение: на телефоне уже должно быть установлено приложение-аутентификатор. 3. Если сотрудник нажимает Отмена , ничего страшного не происходит: вход по логину и паролю не меняется. 4. QR-код создается только после осознанного подтверждения Приложение установлено, продолжить . 5. MFA считается подключенной только после правильного 6-значного кода. 6. После подключения нужно сохранить резервные коды. Они показываются один раз и нужны, если телефон потерян или приложение удалено.

Администратор может выборочно включать требование MFA в разделе Пользователи . Важное правило: Foreman не позволяет сделать MFA обязательной для сотрудника, пока этот сотрудник не подключил приложение-аутентификатор. Это защита от ситуации, когда человек нажал не туда, не установил приложение и запер сам себя.

Дополнительно Foreman ведет audit-контур известных устройств и риск-сигналов входа. Устройство узнается по серверному hash и расширенному цифровому слепку браузера. В слепок входят легкие технические признаки: язык браузера, часовой пояс, размеры экрана и рабочей области, глубина цвета, платформа, наличие touch-ввода, число логических потоков, объем памяти браузера, признаки мобильного режима и похожие параметры. Foreman не использует WebGL/GPU, canvas-аудио, историю посещений, расширения браузера, движения мыши или нажатия клавиш.

Эти данные не дают доступ сами по себе: они помогают понять, знакомый это браузер или новый, изменился ли привычный профиль устройства, были ли странные попытки входа, нужно ли в будущем осторожно попросить дополнительный код. Сейчас риск-сигналы используются как наблюдение и журнал, а не как автоматический бан человека.

Технические заголовки сайта

Foreman Web не должен без необходимости рассказывать внешнему миру, на каком именно серверном фреймворке он работает. Поэтому наружный заголовок X-Powered-By: Express отключен.

Это небольшая, но правильная мера гигиены безопасности: она убирает лишнюю подсказку для автоматических сканеров и поверхностной разведки. При этом ее не нужно переоценивать. Реальную защиту дают роли, cookie-сессия, CSRF, rate-limit, bot-defense, проверка файлов, журналы и тесты; скрывание X-Powered-By только уменьшает ненужное раскрытие технических деталей.

Читать всем

Защита входа от подбора пароля

Foreman Web защищает вход от многократного подбора пароля. Это не мешает обычной работе, но останавливает ситуацию, когда кто-то много раз подряд пробует разные пароли.

Как это выглядит для пользователя:

- несколько обычных ошибок при вводе пароля допускаются;
- если один и тот же логин слишком много раз вводит неверный пароль, вход по этому логину временно ставится на паузу;
- если с одного нормализованного IP идет слишком много неверных попыток, включается дополнительная мягкая защита от атаки;

- уже вошедших пользователей эта защита не выбрасывает из кабинета;
- сайт не сообщает, существует ли такой логин, чтобы не помогать подбору учетных записей.

Если появилось сообщение Слишком много неверных попыток входа. Попробуйте позже. , нужно подождать около 10 минут и попробовать снова. Если пароль забыт, лучше использовать восстановление доступа или обратиться к администратору, а не продолжать угадывать.

Отдельное важное уточнение: Foreman не считает клиентский заголовок X-Forwarded-For доказательством IP-адреса для блокировок. Подставленные снаружи X-Forwarded-For , X-Real-IP , CF-Connecting-IP и Forwarded не должны создавать отдельный лимит и не должны обходить защиту входа. Пока проху-цепочка Amvera не оформлена как доверенная инфраструктурная настройка, такие заголовки используются только как диагностика, если вообще доходят до backend.

Читать всем

Права на заявки

Действие	Кто может
Создать черновик	Не гость
Смотреть журнал	Авторизованный пользователь, включая гостя
Открыть карточку	Авторизованный пользователь, включая гостя
Редактировать заявку	Автор или пользователь с разрешенными правами; обычно только черновик
Отправить заявку	Автор, если заявка находится в допустимом состоянии
Удалить заявку	Автор или администратор, пока у заявки нет офисного статуса
Менять вложения	Автор черновика
Смотреть чужую заявку	Можно, если раздел доступен
Править чужую заявку	Нельзя

Смысл для пользователя простой: если заявка уже ушла в работу или принадлежит другому человеку, кнопки могут быть неактивны. Это защита, а не поломка.

Читать всем

Права на документохранилище

Действие	Обычный пользователь	Гость
Смотреть Мои документы	Да	Нет
Смотреть Общие документы	Да	Да
Скачать из общих документов	Да	Да
Создать папку	Да	Нет
Переименовать папку	Да	Нет
Удалить папку	Да	Нет
Загрузить файлы	Да	Нет
Переименовать файл	Да	Нет
Удалить файл	Да	Нет
Переместить/копировать	Да	Нет

Действие	Обычный пользователь	Гость
Прикрепить к заявке	Да, при наличии прав на заявку	Нет
Отправить на e-mail	Да	Нет

Гость на сервере ограничен общими документами: попытка открыть личный раздел или выполнить управляющее действие должна быть отклонена.

Ограничения файлов в вебе

В документохранилище загрузка ограничена:

- до 20 файлов за одну загрузку;
- до 25 МБ на один файл.

Для групповых действий в документохранилище действуют лимиты:

- прикрепить, переместить или копировать - до 30 файлов за действие;
- отправка на e-mail - до 30 файлов и до 20 адресатов.

Эти ограничения нужны, чтобы случайная массовая операция не перегрузила приложение и не отправила лишнее.

После этих проверок файл проходит антивирусный контур Foreman. Для пользователя это выглядит просто: обычный рабочий файл быстро сохраняется и получает отметку **Проверка Касперского не завершена**, а сама проверка Kaspersky продолжается в фоне. Когда ответ готов, отметка меняется на зеленую **Kaspersky: угроз не найдено** или на предупреждение/опасность. Если Kaspersky не дал окончательный зеленый ответ, но и не сообщил об угрозе, файл не теряется: Foreman сохраняет его с предупреждающей отметкой **риск**, а скачивание требует осознанного подтверждения. Это правило относится ко всем разрешенным рабочим форматам. Если файл просто переименовали под документ, фото, текстовый файл или чертеж, Foreman не принимает его как настоящий рабочий файл. Если файл опасный, подозрительный или запрещенный по типу, загрузка блокируется сразу и не сохраняется.

Скачивание файлов

Скачивание в вебе также идет через авторизованный запрос. Нельзя просто взять закрытую ссылку на файл и открыть ее без действующей сессии.

Практическое правило: если файл скачался у вас, это не значит, что его можно пересылать кому угодно. Доступ в Foreman и право пересылки - разные вещи.

Что делать на общем компьютере

1. Не сохранять пароль в чужом браузере.
2. После работы нажать `Выйти`.
3. Не оставлять вкладку Foreman открытой.
4. Не передавать гостевой доступ человеку, которому нельзя видеть заявки.
5. Не загружать личные документы в `Общие документы`.

Справочно

В.9 Безопасность в мобильном приложении Foreman

Читать всем

Хранение входа

Мобильное приложение хранит сессию в защищенном хранилище Flutter. На Android используется `encryptedSharedPreferences`.

Что это значит:

- токен не лежит как простой текст в обычных настройках приложения;
- приложение само добавляет `Authorization: Bearer ...` к запросам;
- при ответе `401` локальная сессия сбрасывается;
- при выходе из аккаунта локальная сессия удаляется.

Локальные черновики

Мобильная версия умеет работать с локальными черновиками и синхронизацией. Это удобно на объекте, но добавляет ответственность за телефон. В актуальной APK офлайн-черновики и очередь отправки сохраняются в защищенном хранилище приложения; старые локальные данные из обычных настроек приложения автоматически переносятся в защищенное хранилище и удаляются из прежнего места.

Правила:

- не удалять приложение, если есть несинхронизированные черновики;
- не давать телефон с открытым Foreman постороннему человеку;
- включить блокировку экрана телефона;
- после плохой связи проверить, что черновик дошел до сервера;
- если телефон потерян, администратор должен заблокировать доступ Foreman Mobile на этом смартфоне, а при подозрительных действиях дополнительно сменить пароль или отключить аккаунт.

Фото и файлы с телефона

Приложение использует камеру и галерею только когда пользователь сам выбирает соответствующее действие, например `Камера` или `Галерея`.

Практическое правило: перед отправкой заявки проверить, что фото относится именно к этой заявке и не прикреплено случайно.

Фото и файлы с телефона уходят через тот же сервер Foreman, что и веб-браузер. Поэтому они также проходят антивирусный контур перед привязкой к заявке. Пользователю не нужно запускать отдельную проверку на телефоне: если файл принят, он прошел серверную проверку; если заблокирован, это будет видно администратору в журнале проверок.

Сервер и связь

Раздел `Сервер и связь` нужен для диагностики. В актуальной рабочей APK серверный адрес зафиксирован на утвержденный Foreman API, а произвольная смена сервера в production не является обычной пользовательской функцией.

Важно: сервисный PIN не заменяет логин и пароль. Он только защищает локальную настройку сервера от случайного изменения.

Подпись и идентичность APK

Актуальная Android-сборка Foreman Mobile выпускается не как временное debug-приложение, а как production APK с постоянной подписью, нормальным applicationId `ru.tzertis.foreman` и включенным obfuscation.

Для пользователя это означает простую вещь: телефон видит Foreman как одно и то же приложение от выпуска к выпуску, а обновление не должно внезапно превращаться в "другое похожее приложение". Кроме того, Android backup для приложения отключен, чтобы системная резервная копия телефона не переносила локальные данные Foreman туда, где они не нужны.

Это не делает мобильное приложение "идеальным банковским уровнем". Честный остаток риска остается: APK публикуется на Amvera вручную, поэтому важны аккуратное имя файлов, наличие комплекта `foreman-release.apk`, `app-update.json`, `app-update.json.sig` и контрольная установка на реальном устройстве после выпуска.

Обновления APK

Мобильное приложение проверяет обновления через настроенный манифест. Перед доверием к `app-update.json` приложение проверяет подпись `app-update.json.sig`; затем скачивает APK через Android DownloadManager в стандартную папку «Загрузки» и сверяет SHA-256 именно этого файла с тем значением, которое указано в подписанном манифесте обновления. Если манифест, файл APK или контрольная сумма подменены, повреждены или загружены не полностью, установка не должна продолжаться как обычное обновление.

Безопасное правило:

- ставить APK только из доверенной ссылки Foreman;
- не устанавливать "похожий Foreman" из случайного сообщения;
- если обновление не ставится, обратиться к администратору;
- перед переустановкой убедиться, что черновики синхронизированы.

Если телефон потерян или сотрудник уволился

Администратор должен:

1. В разделе доверенных устройств найти мобильное устройство сотрудника.
2. Нажать **Заблокировать устройство**, если нужно отключить доступ Foreman Mobile именно на потерянном смартфоне.
3. Проверить, что активные мобильные сессии этого устройства отозваны.
4. Проверить, не было ли подозрительных действий.
5. Если сотрудник уволился, пароль утек или APK вместе с логином и паролем ушел посторонним людям, нажать **Отключить доступ сотрудника**, а не просто очищать цифровые слепки.
6. Выдать новый доступ на новый телефон.

Что именно делает блокировка мобильного устройства:

- ставит устройству статус **заблокировано**;
- отзывает активные серверные сессии этого экземпляра Foreman Mobile;
- не дает этому экземпляру Foreman Mobile войти повторно;
- не блокирует другие устройства сотрудника и не отключает самого сотрудника целиком.

Что делает **Отключить доступ сотрудника**:

- деактивирует учетную запись без удаления заявок, документов и истории;
- отзывает все активные web- и mobile-сессии сотрудника;
- блокирует известные мобильные устройства и профили этого сотрудника;
- не стирает рабочую историю, чтобы аудит и документы не исчезли вместе с человеком;
- делает APK бесполезным для этого логина, даже если файл приложения, логин и пароль оказались у посторонних.

Пользователь должен:

1. Сообщить администратору как можно быстрее.
2. Не ждать “до завтра”, если в телефоне был открытый Foreman.

Читать всем

В.10 Где смотреть безопасность Telegram-бота

Для Telegram-бота безопасность раскрыта внутри **Раздела 3**:

- **3.1 Роли и права**;
- **3.6 Файлы заявок**;
- **3.7 Карточка заявки в Telegram**;
- **3.10 Удаление и восстановление**;
- **3.18 Админские команды**;
- **3.19 Устойчивость к паузам и ошибкам**.

Эти правила не повторяются здесь полностью, чтобы не было дублирования. Смысл тот же: права проверяются по роли, привязке Telegram-пользователя к ФИО и принадлежности заявки.

Раздел 1. Foreman Web в браузере

Foreman Web - основной экран для работы на ПК. Здесь удобнее всего вести журнал, документы, таймлайн, план-график, карточки и вложения.

1.1 Верхняя панель

После входа сверху видны:

Кнопка / область	Что делает
Foreman Web	Возвращает в основной контур
Имя пользователя и должность	Открывает профиль при нажатии
Новая заявка	Открывает выбор типа новой заявки
Таймлайн	Открывает график движения заявок
План-график	Открывает интерактивный план запусков по объектам, датам и статусам
Документохранилище	Открывает файлы и папки Foreman
↻	Обновляет текущие данные
Выйти	Завершает сессию пользователя

Активная верхняя кнопка подсвечивается. Например, когда открыт таймлайн, кнопка Таймлайн должна быть визуально активной. Когда открыт план-график, активной должна быть кнопка План-график .

Вход, восстановление доступа и срок сессии описаны в общем блоке Перед началом : пункты В.1 , В.3 , В.7 , В.8 . В web-разделе ниже описана ежедневная работа после входа.

1.2 Левая колонка в журнале

В журнале слева находятся:

Элемент	Назначение
Журнал	Список отправленных и рабочих заявок
Черновики	Заявки, которые еще не отправлены
Поиск	Поиск по списку
Все типы	Фильтр по типу заявки
Все статусы	Фильтр по статусу
список заявок	Быстрый выбор заявки

У гостя вкладка Черновики не показывается.

1.3 Как открыть заявку

Порядок:

1. Открыть **Журнал**. 2. Найти заявку в списке слева. 3. Нажать на строку заявки. 4. В центре откроется карточка заявки. 5. Справа откроются панели **Вложения**, **Внешние вложения**, **Журнал доставки**.

Что проверять:

- номер заявки;
- объект;
- статус;
- составитель;
- даты;
- вложения;
- доступные кнопки.

1.4 Кнопки в карточке заявки

Кнопка	Когда полезна	Ограничение
Открыть в редакторе	Вернуться к редактированию	Доступно не всегда, зависит от статуса и прав
Отправить себе	Получить заявку себе на e-mail	Нужен e-mail и право отправки
Удалить	Удалить черновик или доступную заявку	Для чужих/отправленных заявок может быть запрещено
Preview	Предпросмотр для оконного блока	Для некоторых типов отключена
PDF	Сформировать или скачать PDF	Может занять время
DOCX	Сформировать DOCX	Для некоторых типов отключена

Если кнопка бледная или не нажимается, действие сейчас недоступно. Это не всегда ошибка.

1.5 Preview, PDF, DOCX и отправка на e-mail

В карточке и редакторе есть два разных сценария, которые важно не путать:

Сценарий	Что делает
Preview	Показывает визуальный предпросмотр, где это поддерживается
PDF / Сформировать PDF	Собирает готовый PDF-документ заявки
DOCX	Собирает Word-документ, если тип заявки это поддерживает
Отправить на e-mail	Открывает отправку готовой заявки или файла адресатам
Отправить себе	Быстро отправляет выбранную заявку на e-mail текущего пользователя

Практическая последовательность:

1. Сначала заполнить и сохранить заявку. 2. Для оконного блока или переделки дождаться готового preview/PDF. 3. Проверить, что документ сформирован без ошибки. 4. Нажать **Отправить на e-mail**. 5. Выбрать адресатов из списка или ввести адрес вручную. 6. Нажать отправку и дождаться сообщения об успехе.

Важно: e-mail-отправка доступна не гостю и требует корректного адреса. Если кнопка отключена, чаще всего еще не сформирован документ, не заполнены обязательные данные, нет права отправки или у пользователя не указан e-mail.

Для чтения

1.6 Создание новой заявки в вебе

Порядок:

1. Нажать **Новая заявка**. 2. Выбрать тип заявки. 3. Заполнить форму. 4. Нажать **Сохранить черновик** или **Сохранить**. 5. При необходимости прикрепить файлы. 6. Проверить итоговую форму. 7. Нажать **Отправить на e-mail**, если заявка готова.

Типы заявок, которые отображаются в системе:

Тип	Когда использовать
Заявка на заполнения	Материалы/позиции для заполнений
Заявка на расходники	Расходные материалы
Балконный/оконный блок	Оконно-балконные конструкции
Переделка рамы/створки	Переделки, связанные с рамой или створкой

Для чтения

1.7 Черновики в вебе

Черновик - это заявка, которая еще не ушла окончательно.

Порядок:

1. Открыть вкладку **Черновики**. 2. Выбрать нужный черновик. 3. Нажать **Открыть в редакторе**. 4. Исправить данные. 5. Нажать **Сохранить черновик**. 6. Когда готово - отправить.

Практическое правило: если человека отвлекли, сначала сохранить черновик.

Для чтения

1.8 Вложения в заявке

Панель справа называется **Вложения**.

Там есть два способа добавить файл:

Кнопка	Что делает
С компьютера	Выбор файла с текущего ПК
Из документов	Выбор файла из документохранилища Foreman

Добавить файл с компьютера

1. Открыть заявку или черновик. 2. В панели **Вложения** нажать **С компьютера**. 3. Выбрать файл. 4. Дождаться загрузки. 5. Проверить, что файл появился в списке вложений.

Добавить файл из документохранилища

1. Открыть заявку или черновик. 2. В панели **Вложения** нажать **Из документов**. 3. Откроется окно **Добавить из документохранилища**. 4. Выбрать **Мои документы** или **Общие документы**. 5. Открыть нужную папку. 6. Отметить один или несколько файлов чекбоксами. 7. Нажать **Прикрепить к заявке**. 8. Проверить правую панель **Вложения**.

Если в окне не видны названия файлов, это ошибка интерфейса и ее нужно фиксировать скриншотом. В нормальном виде должны быть видны и иконка типа файла, и название.

Для чтения

1.9 Как выглядят вложения

Для файлов используются узнаваемые значки:

Тип	Как должен выглядеть
Word doc/docx	Синий значок с W
Excel xls/xlsx	Зеленый значок с X
PDF	Красный/похожий PDF-стиль
Фото jpg/jpeg/png	Иконка изображения
AutoCAD dwg/dxf	CAD-стиль
Остальное	Нейтральный файловый значок

В правой колонке вложение должно показывать:

- иконку типа;
- название файла;
- тип/расширение;
- размер;
- кнопку **Скачать**;
- кнопку **Убрать**, если файл можно убрать.

Для чтения

1.10 Внешние вложения

Панель **Внешние вложения** показывает файлы, которые связаны с заявкой из внешнего источника или импортированной карточки.

Обычное действие:

1. Открыть карточку заявки. 2. Найти панель **Внешние вложения**. 3. Нажать **Скачать** у нужного файла.

Гость может скачивать доступные внешние вложения, но не должен менять заявку.

1.11 Журнал доставки

Панель **Журнал доставки** показывает сведения по отправлениям и доставке, если они есть.

Кнопка  в этой панели обновляет лог доставки.

Если записей нет, показывается сообщение вроде **Записей доставки нет**.

1.12 Документохранилище

Документохранилище - это файловый раздел Foreman. Он нужен, чтобы не хранить рабочие PDF, Word, Excel, фото и чертежи только на личных компьютерах.

1.12.1 Как открыть

1. В верхней панели нажать **Документохранилище**. 2. Слева выбрать корень: - **Мои документы**; - **Общие документы**. 3. Открыть нужную папку.

У гостя открываются только **Общие документы**.

1.12.2 Левая часть документохранилища

Слева видны:

Элемент	Что делает
Мои документы	Личная зона пользователя
Общие документы	Общая зона документов
папки	Переход внутрь папки
Файлы текущей папки	Быстрый список файлов текущей папки

Активная папка подсвечивается.

1.12.3 Верхние кнопки

Кнопка	Что делает	Кому доступна
К заявкам	Вернуться из документохранилища к заявкам	Всем
Обновить	Перезагрузить список папок и файлов	Всем
Создать папку	Создать папку в текущем месте	Не гостю
Загрузить файлы	Загрузить файлы в текущую папку	Не гостю
Переименовать папку	Переименовать открытую папку	Не гостю
Удалить папку	Удалить папку	Не гостю

1.12.4 Создать папку

1. Открыть нужный раздел. 2. Нажать **Создать папку**. 3. Ввести название. 4. Нажать **Сохранить**.

Если папка не появилась:

- нажать **Обновить**;
- проверить, не открыт ли гостевой аккаунт;
- проверить, не пустое ли название.

1.12.5 Переименовать папку

1. Открыть папку. 2. Нажать **Переименовать папку**. 3. Ввести новое название. 4. Нажать **Сохранить**.

Если кнопка не работает:

- проверить, что открыта именно папка, а не корень;
- проверить, что пользователь не гость;
- обновить страницу;
- сообщить администратору название папки и время ошибки.

1.12.6 Удалить папку

1. Открыть папку. 2. Нажать **Удалить папку**. 3. Подтвердить удаление.

Ограничение: обычная папка может не удалиться, если внутри есть файлы или вложенные папки. Общая папка может удаляться вместе с вложениями только при разрешенном сценарии.

1.12.7 Загрузить файлы

1. Открыть нужную папку. 2. Нажать **Загрузить файлы**. 3. Выбрать один или несколько файлов. 4. Дождаться загрузки. 5. Проверить список файлов.

Поддерживаются рабочие форматы: `jpeg`, `jpg`, `png`, `pdf`, `doc`, `docx`, `xls`, `xlsx`, `txt`, `rtf`, `dwg`, `dxg`.

Файлы-страницы и сценарии (`html`, `htm`, `mhtml`, `hta`, `js`, `vbs`, `lnk`, `scr`, `iso`, `svg`, `zip`) в документохранилище не принимаются. Это не каприз: такие файлы могут быть использованы для атаки через браузер, когда вредный код собирается уже на компьютере сотрудника.

Техническое ограничение web-загрузки: за один раз можно выбрать до 20 файлов, размер одного файла - до 25 МБ. Если файлов больше, лучше загрузить их несколькими заходами.

1.12.8 Поиск по текущей папке

Поле **Поиск по текущей папке** фильтрует файлы внутри открытой папки.

Порядок:

1. Открыть папку. 2. Ввести часть названия файла. 3. Смотреть отфильтрованный список.

Поиск не заменяет навигацию по всем папкам. Он работает по текущей папке.

1.12.9 Отображение файлов

В форме Действия с файлами есть блок Отображение файлов :

Кнопка	Когда использовать
Строки	Когда файлов много и нужно читать названия
Превью	Когда важен тип файла или визуальная карточка

Активный режим подсвечивается.

1.12.10 Сортировка файлов

В форме Действия с файлами есть блок Сортировка файлов :

Кнопка	Что делает
Сортировка по имени	Сортирует по названию файла
Сортировка по типу	Группирует/упорядочивает по расширению и типу
Сортировка по дате изменения	Показывает порядок по последнему изменению

Активная сортировка подсвечивается. Повторное нажатие может менять направление сортировки.

1.12.11 Выбор файлов

Файлы выбираются чекбоксами.

После выбора справа становятся доступны действия:

Действие	Условие
Просмотр	Обычно нужен один выбранный файл
Скачать	Обычно нужен один выбранный файл
Переименовать	Один выбранный файл, не гость
Удалить выбранные	Один или несколько файлов, не гость
Переместить	Выбраны файлы и выбрана папка назначения
Копировать	Выбраны файлы и выбрана папка назначения
Прикрепить к заявке	Выбраны файлы и выбрана заявка
Отправить файлы	Введен e-mail и выбраны файлы
Сбросить выбор	Есть выбранные файлы

Для перемещения, копирования, прикрепления к заявке и e-mail-отправки не стоит выбирать слишком большие пачки. Рабочее ограничение таких массовых действий - до 30 файлов за операцию.

1.12.12 Прикрепить файл из документохранилища к заявке

1. Открыть **Документохранилище** . 2. Найти файл. 3. Отметить файл чекбоксом. 4. В правой форме выбрать заявку в поле **Выберите заявку** . 5. Нажать **Прикрепить к заявке** . 6. Открыть журнал и проверить вложение в карточке.

Минимум кликов для прораба: если он уже в заявке, лучше нажать **Из документов** прямо в панели **Вложения** .

1.12.13 Отправить файлы на e-mail

1. Выбрать один или несколько файлов. 2. В блоке **Отправить на e-mail** ввести адрес. 3. Нажать **Отправить файлы** .

Если несколько адресов, использовать формат, который принимает поле: через запятую или с новой строки.

По необходимости 1.12.14 Гость в документохранилище

Гость видит только **Общие документы** .

Разрешено:

- открыть папку;
- переключить **Строки** / **Превью** ;
- сортировать;
- отметить файл;
- открыть просмотр;
- скачать.

Запрещено:

- загрузить;
- создать папку;
- переименовать;
- удалить;
- переместить;
- копировать;
- прикрепить к заявке;
- отправить по e-mail.

1.13 Таймлайн

Таймлайн показывает движение заявок по датам и статусам.

1.13.1 Как открыть

1. В верхней панели нажать **Таймлайн**. 2. Дождаться загрузки графика.

1.13.2 Период по умолчанию

По умолчанию:

Направление	Значение
Назад	11 дней
Вперед	2 дня

Это сделано потому, что для контроля важнее последние рабочие дни, чем длинный будущий период.

1.13.3 Кнопки и элементы таймлайна

Элемент	Что делает
Назад: ... дн.	Ползунок количества дней назад
Вперёд: ... дн.	Ползунок количества дней вперед
Сегодня	Центр периода
Обновить	Перезагружает данные
Скачать PDF	Формирует PDF-отчет
цветные статусы	Показывают этапы движения
бледно-желтый фон	Выходные дни

1.13.4 Как пользоваться

1. Открыть таймлайн. 2. Посмотреть период. 3. Если нужно, передвинуть ползунки. 4. Нажать **Обновить**. 5. Найти заявки с долгими цветными участками. 6. При необходимости нажать **Скачать PDF**.

1.13.5 Что смотреть руководителю

Смотреть не только “какая заявка есть”, а сколько времени она висит в статусе.

Признаки проблемы:

- длинный красный участок;
- заявка несколько дней без движения;
- много заявок в одном статусе;
- дата отправки есть, а следующего этапа нет.

1.14 План-график

План-график - это интерактивная рабочая доска по объектам, датам, карточкам запусков и фактическим этапам работы. Его смысл не в том, чтобы один человек "нарисовал красивую таблицу", а остальные смотрели на нее как на стенд. План-график нужен для командной работы: каждый участник отмечает свой участок ответственности, а вся команда сразу видит живую картину без звонков, пересылок и ручного сведения разных версий.

Главное преимущество план-графика в том, что он двусторонний. Руководитель видит план, офис видит, что уже заведено в Альтавин, производство и объект видят движение дальше, а прораб не остается в тумане "кто-то там занимается". Если один сотрудник изменил разрешенный ему статус, остальные видят эту обратную связь практически в режиме реального времени после обновления данных.

1.14.1 Как открыть

1. Войти в Foreman Web. 2. В верхней панели нажать **План-график**. 3. Дождаться загрузки объектов, строк, дат и карточек.

Если кнопка не видна или действие недоступно, причина обычно в правах пользователя или в том, что для вашей роли раздел пока открыт только на просмотр.

1.14.2 Что видно на экране

План-график собирает в одном месте:

- объекты;
- строки по объектам и ответственным;
- календарные ячейки;
- карточки запусков;
- секции, этажи, количество изделий и площадь;
- текущие статусы;
- служебные отметки вроде исполнительных схем и паспортов качества;
- примечания внутри карточек;
- инфо-плашки в ячейках;
- форму **Хронология заявок**;
- историю изменений.

Карточка в план-графике - это не просто цветной прямоугольник. Это рабочий сигнал: где стоит запуск, к какому объекту он относится, что с ним уже произошло и какой следующий этап ожидается.

1.14.3 Нужно ли самому разлиновывать график

Нет. План-график не нужно разлиновывать вручную, как Excel.

Пользователь не рисует колонки, не протягивает даты, не копирует формулы и не делает руками сетку на следующий месяц. Foreman Web сам строит календарные колонки, строки объектов и ячейки на выбранный период. Вы открываете раздел, выбираете нужный диапазон дат, а веб-браузер автоматически показывает готовую рабочую сетку.

Это важное отличие от обычной таблицы. В Excel человек сначала тратит время на “подготовку листа”: нарисовать шапку, продлить даты, проверить выходные, не сбить ширину колонок. В план-графике эта техническая работа уже заложена в программу. Пользователь занимается содержанием: добавляет карточки, переносит запуск, ставит разрешенные статусы, пишет примечания и инфо-плашки.

Технически график может строиться на годы вперед, потому что даты создаются программно. Практически удобнее работать разумными периодами: неделя, месяц, квартал или сезон. Очень длинный период на несколько лет лучше открывать не “одним бесконечным полотном”, а частями, иначе экран станет слишком широким и тяжелым для восприятия.

Простое правило: если нужно планировать дальше, не рисуйте новые колонки сами. Просто выберите или пролистайте нужный период, а Foreman построит даты автоматически.

1.14.4 Значки + и T в ячейке

В нижнем правом углу ячейки могут быть маленькие рабочие значки. Они нужны, чтобы быстро добавить информацию прямо в нужный день и нужное место графика.

Значок	Для чего нужен
	Создать новую карточку запуска в этой ячейке
	Добавить текстовую инфо-плашку в этой ячейке

 используют, когда нужно поставить в план новый запуск: выбрать объект, дату и строку, а затем заполнить данные карточки.

 используют не для заявки, а для поясняющей информации по ячейке. Это удобно, когда в этот день на объекте есть обстоятельство, которое влияет на план, но не является отдельной заявкой.

1.14.5 Инфо-плашки и заглушки

Инфо-плашка - это текстовая заметка прямо в ячейке план-графика. Ее можно использовать как аккуратную “заглушку” или предупреждение для команды.

Примеры:

-  Демонтаж крана ;
-  Работы не ведутся ;
-  Ожидаем доступ на этаж ;
-  Перенос из-за поставки ;
-  Бригада на другом участке .

Такие плашки помогают объяснить, почему в ячейке нет обычной карточки запуска или почему план на день выглядит иначе. Это лучше, чем держать важное уточнение в чате: в чате оно быстро уезжает вверх, а в план-графике остается на своем месте, рядом с датой и объектом.

1.14.6 Примечание внутри карточки

Если в верхнем правом углу карточки виден красный квадратик, это сигнал: внутри карточки есть примечание.

Красный квадратик нужен, чтобы важная информация не пряталась. Даже если карточка маленькая и весь текст не помещается на экране, пользователь сразу понимает: карточку стоит открыть и прочитать комментарий.

Примечания полезны для уточнений вроде причины переноса, особенностей секции, договоренности с объектом, замечаний по замерам или другой информации, которую нельзя потерять при движении карточки по графику.

1.14.7 Хронология заявок

В план-графике ведется форма **Хронология заявок**. Она показывает, что происходило с карточками и статусами: кто добавил карточку, кто передвинул ее, кто изменил статус или оставил важное примечание.

Хронология нужна не для поиска виноватых, а для нормальной рабочей памяти системы. Если через несколько дней возник вопрос “почему карточка переехала на другую дату” или “когда поставили этот статус”, не нужно вспоминать по переписке. Достаточно открыть хронологию и посмотреть последовательность событий.

Перенос карточки также фиксируется в хронологии. В записи видно, кто перенес карточку, когда это произошло и с какой даты на какую дату она была переставлена. Это удобно для спокойной рабочей проверки: план меняется, но след изменения не теряется.

Читать всем

1.14.8 Статусы и права доступа

Статусы можно ставить только в пределах своего уровня доступа. Это важная защита, а не придирка системы.

Например:

- сотрудник, который заводит заявки в Альтавин, может отметить этап **Замеры заведены в Альтавин**;
- сотрудник производства может отметить готовность изделий, если ему выданы такие права;
- прораб или ответственный по объекту может отмечать объектовые этапы, например **Изделия смонтированы**, если это входит в его доступ;
- администратор или пользователь с расширенными правами может видеть и настраивать больше, чем обычный сотрудник.

Foreman не дает человеку случайно “переписывать” чужую работу. Если сотрудник не имеет отношения к этапу, он не сможет поставить или снять этот статус. Те, кто заводят заявки в Альтавин, не должны случайно поставить **Изделия смонтированы**; а те, кто отвечает за монтаж, не должны стирать офисную отметку о заведении заявки. Так план-график остается общей картиной, а не местом, где каждый может нечаянно испортить чужой участок.

Если кнопка статуса неактивна или система пишет, что нет прав, это означает: текущему пользователю не назначен этот этап. Нужно либо работать со своими разрешенными статусами, либо обратиться к администратору, если права действительно надо изменить.

1.14.9 Перетаскивание карточек

Карточки можно переносить мышью:

1. Наведите курсор на карточку. 2. Нажмите левую клавишу мыши и удерживайте ее. 3. Перетащите карточку в нужную ячейку по этому же объекту. 4. Отпустите левую клавишу мыши.

Так можно быстро переставить запуск на другую дату или в другую строку внутри объекта. Это удобно, когда план меняется в течение дня: не нужно открывать длинную форму ради каждого небольшого переноса.

После переноса карточка на время выделяется желтой штриховой рамкой. Это визуальный сигнал для команды: карточку недавно переставили, на нее стоит обратить внимание. Подсветка не висит постоянно и не засоряет график - она автоматически исчезает через 2 дня после переноса. Сам факт переноса при этом остается в Хронологии, поэтому даже после исчезновения рамки можно открыть историю карточки и понять, кто и когда изменил дату.

Есть важное ограничение: программа не позволит перетащить карточку в другой объект. Если карточка относится к одному объекту, она должна оставаться внутри этого объекта. Это защищает график от случайных переносов “не туда”, особенно когда на экране много строк и похожих дат.

1.14.10 PDF, печать и e-mail-рассылка

План-график можно подготовить к печати в виде PDF-файла. Это удобно, когда нужно зафиксировать выбранный период, отправить его руководителю, передать на объект или сохранить как рабочий отчет.

При формировании PDF выбирается диапазон дат: например, несколько дней, неделя, месяц или другой нужный период. В PDF попадают только те карточки и ячейки, которые относятся к выбранному диапазону.

Одновременно с формированием PDF можно отправить этот файл на выбранные e-mail адреса. Это экономит время: не нужно сначала скачать файл, потом отдельно открывать почту, вручную прикреплять PDF и рассылать его нескольким людям. Foreman делает это из одного сценария.

Перед отправкой полезно посмотреть дополнительное окно-подсказку. В нем показывается, какие номера приоритетов попадут в выбранный диапазон дат. Это помогает быстро проверить себя до рассылки: например, убедиться, что в PDF действительно попали нужные запуски, а лишний период случайно не захвачен.

Практический порядок такой:

1. Открыть **План-график**. 2. Выбрать нужный диапазон дат. 3. Открыть формирование PDF/печать. 4. Посмотреть в окне, какие номера приоритетов попадают в этот период. 5. Если все верно, сформировать PDF. 6. При необходимости выбрать e-mail адресатов и отправить PDF сразу из Foreman.

Если в окне с приоритетами видно, что нужных номеров нет или попали лишние, не отправляйте PDF сразу. Сначала поправьте диапазон дат, обновите просмотр и только потом формируйте файл.

1.14.11 Возможная связь с кладочными планами

В дальнейшем план-график можно развивать дальше и связать его с графиками кладочных планов, согласованных с Заказчиком. Это позволит сопоставлять внутренний производственный план с тем, что уже принято и согласовано по объекту.

Идея простая: если кладочный план утвержден Заказчиком, его график может стать внешним ориентиром для планирования запусков, замеров, производства и монтажа. Тогда команда будет видеть не только свои внутренние карточки, но и связь с согласованной строительной логикой объекта.

Такой подход помогает заранее замечать расхождения: например, когда внутренний план запуска уходит вперед или назад относительно согласованного кладочного графика. Это не заменяет работу руководителя, но дает ему более удобную основу для решений и переговоров.

Важно: этот пункт описывает направление развития. Текущий план-график уже можно использовать как самостоятельный рабочий инструмент, а связь с кладочными планами может стать следующим уровнем интеграции.

1.14.12 Почему это лучше обычной таблицы

Обычная таблица часто работает в одну сторону: один человек заполнил, остальные посмотрели, потом начались звонки, уточнения и версии “а у меня файл свежее”. Интерактивный план-график работает иначе. Он превращает план в общую рабочую поверхность.

Преимущество в том, что каждый видит не только план, но и реакцию команды на этот план:

- офис отмечает свои этапы;
- производство показывает готовность;
- объект видит, что можно ожидать дальше;
- руководитель видит узкие места до того, как они превратились в пожар;
- история помогает понять, кто и когда изменил карточку или статус.

Это не контроль ради контроля. Это способ убрать лишний шум из работы: меньше ручных сверок, меньше “я думал, что уже сделано”, меньше потерянных договоренностей. Чем аккуратнее команда пользуется статусами и карточками, тем полезнее становится план-график для всех.

1.14.13 Что делать, если изменение не сохраняется

Проверьте простые вещи:

- есть ли интернет и открывается ли Foreman Web;
- не пытаетесь ли вы поставить статус, который не входит в ваши права;
- не переносите ли карточку в другой объект;
- не устарели ли данные на экране, если другой сотрудник уже изменил эту карточку.

Если сомневаетесь, нажмите **Обновить** и повторите действие. Если запрет повторяется, лучше не обходить систему, а передать ситуацию администратору: чаще всего нужно не “сломать защиту”, а правильно настроить права.

Администратору

1.15 Профиль и администрирование

По необходимости

1.15.1 Профиль пользователя

Открытие:

1. Нажать на имя пользователя в верхней панели. 2. Откроется профиль.

В профиле важны:

- ФИО;
- e-mail;

- телефон;
- должность;
- роль.

Если человека нельзя называть прорабом, нужно исправить поле **Должность**.

1.15.2 Web Push-уведомления

Web Push - это короткое уведомление от Foreman, которое браузер может показать на компьютере или смартфоне. Оно не заменяет e-mail, Telegram и рабочую карточку заявки. Его задача другая: быстро подсветить событие, на которое человеку стоит обратить внимание.

Пример нормального push:

Foreman: заявка ЖА-77
получена чат-ботом и ожидает обработки.

Пользователь включает push сам:

1. Нажать на свое ФИО в верхней панели. 2. Открыть **Мои реквизиты**. 3. В блоке **Push-уведомления** нажать **Включить push**. 4. Разрешить уведомления в браузере, если браузер спросит.

Если браузер или телефон не дали разрешение, Foreman не сможет показать push на этом устройстве. Это не поломка заявки: письмо, журнал, таймлайн и карточка остаются основными источниками правды. Push - быстрый сигнальный огонек, а не единственный канал управления.

Администратор отдельно назначает, по каким событиям и каким сотрудникам отправлять push. Для этого в профиле администратора есть таблица **Push-уведомления сотрудников**. В ней выбирается сотрудник и отмечаются события, например:

- **ALARM по вирусу** ;
- **Заявка получена ботом и ждет офис** ;
- **Заявка не заведена 24 часа** ;
- **Подключение руководства** ;
- **План-график: изделия готовы** ;
- **План-график: изделия вывезены на объект** .

Практическое правило: push отправляется только сотрудникам, у которых включены уведомления в браузере. Если сотрудник не включил push на своем телефоне или компьютере, галочка в админской таблице сама по себе не заставит устройство принимать уведомления.

Тестовая отправка push спрятана в админской зоне и нужна только для проверки настройки конкретного сотрудника. Обычным пользователям она не нужна: им достаточно кнопки **Включить push**.

1.15.3 Защита входа от ботов

В Foreman Web добавлена дополнительная защита входа `honeypot/bot-defense`. Сейчас она стоит именно на форме входа. В журнале заявок, план-графике, таймлайне и документохранилище работают другие защиты: роли, права, ограничения гостя, серверные проверки, `proxy-allowlist`, журналы и антивирусный контур.

Идея `honeypot` простая и довольно красивая: человеку показывается обычная форма входа, а для бота в форме есть скрытая ловушка. Сотрудник ее не видит, не трогает и спокойно вводит логин с паролем. Автоматическая программа может заполнить скрытое поле или отправить форму слишком механически. Тогда Foreman не ругается загадочной ошибкой, а записывает событие в журнал `bot-defense`.

Чем это отличается от ограничения по IP:

Защита	На что смотрит	Простой пример
Ограничение по IP	На количество действий с одного адреса	Один адрес слишком часто вводит неверный пароль или слишком много скачивает
Ограничение по логину	На подбор пароля к конкретному пользователю	К логину Оксаны много раз подряд вводят неверный пароль
Honeypot	На способ отправки формы входа	Форма отправлена так, как ее обычно отправляет программа, а не живой человек

Главное правило: один сигнал не блокирует человека. Foreman не должен превращаться в систему, которая из-за одного странного клика не пускает сотрудника на работу. Временная блокировка возможна только осторожно - при повторном автоматическом поведении или при сочетании с другими факторами, например большим числом неверных паролей.

Администратор может смотреть журнал `bot-defense`, видеть активные временные блокировки, снять блокировку вручную и при необходимости перевести защиту в режим `только журнал`. Это аварийный предохранитель: если защита начала мешать живым людям, ее можно быстро сделать наблюдающей, не переписывая код.

Честно: `honeypot` не делает сайт "невзламываемым" и не заменяет внешний WAF/anti-DDoS, независимый пентест и круглосуточный мониторинг. Но для текущего масштаба Foreman это полезный слой рядом с MFA: он снижает риск простого автоматического подбора и дает администратору ранние признаки, что форму входа пытается использовать программа.

1.15.4 Двухфакторная защита входа MFA

MFA - это дополнительная проверка входа. Пароль остается первым ключом, а одноразовый код из приложения становится вторым. Если пароль кто-то узнал случайно, без кода из приложения войти будет намного сложнее.

Для Foreman подходят приложения-аутентификаторы: Яндекс Ключ, Google Authenticator, Microsoft Authenticator, Aegis, 2FAS и другие TOTP-приложения. Это не привязка к Google и не платная SMS-рассылка.

Почему выбран TOTP, а не SMS:

- код не зависит от доставки SMS и оператора связи;
- код можно получить даже при слабой мобильной сети, если приложение уже настроено;
- нет расходов на отправку SMS;
- ниже риск атаки через перевыпуск SIM-карты;
- код живет короткое время и создается прямо на телефоне сотрудника.

Как сотруднику подключить MFA:

1. Установить на смартфон Яндекс Ключ, Google Authenticator, Microsoft Authenticator, Aegis, 2FAS или другое приложение для одноразовых кодов. 2. В Foreman Web нажать свое ФИО в верхней панели. 3. Открыть Мои реквизиты. 4. В блоке Двухфакторная защита входа нажать Подключить MFA. 5. Прочитать предупреждение. Если приложения на телефоне еще нет, нажать Отмена. 6. Если приложение установлено, нажать Приложение установлено, продолжить. 7. Отсканировать QR-код телефоном. 8. Ввести 6 цифр из приложения. 9. Сохранить резервные коды в надежном месте.

Важно для сотрудников: если нажать Подключить MFA случайно, вход не блокируется. Foreman сначала показывает предупреждение. QR-код создается только после второго осознанного подтверждения, а MFA включается только после правильных 6 цифр. Если вы не уверены, нажмите Отмена и попросите администратора помочь.

Важно для администратора: в разделе Пользователи можно выборочно сделать MFA обязательной, но только для тех, кто уже подключил приложение-аутентификатор. Это сделано специально, чтобы не запереть человека без подготовленного телефона. При потере телефона администратор должен сбросить MFA и помочь сотруднику подключить приложение заново; для аварийной ситуации предусмотрено временное отключение MFA через переменную на Amvera.

По необходимости 1.15.5 Пользователи

Раздел Пользователи доступен администратору.

Администратор может:

- создать пользователя;
- изменить ФИО;
- изменить логин;
- изменить e-mail;
- изменить телефон;
- изменить должность;
- изменить роль;
- включить пользователя или отключить доступ сотрудника;
- посмотреть, подключена ли MFA;
- сделать MFA обязательной для сотрудника, который уже настроил приложение-аутентификатор;
- выдать код доступа.

Для увольнения, утечки логина/пароля, потерянного телефона или другого административного решения используется не "удаление сотрудника", а отдельное действие Отключить доступ сотрудника. Оно оставляет заявки, документы и историю на месте, но закрывает человеку дорогу обратно: учетная запись становится неактивной, активные web/mobile-сессии отзываются, а известные устройства и мобильные профили переводятся в заблокированное состояние. В журнале остается событие с причиной и администратором, который выполнил действие.

Важно не путать это с кнопкой **Очистить цифровые слепки устройств**. Очистка цифровых слепков нужна для сброса диагностических профилей браузеров и телефонов. Она не увольняет сотрудника, не отзывает его сессии и не должна использоваться как мера безопасности при компрометации пароля.

Справочно

1.15.6 Безопасность: тревоги, события и отчеты

В админской части Foreman есть раздел **Безопасность**. Его задача не в том, чтобы пугать администратора красными словами, а в том, чтобы важные следы не растворялись в обычном рабочем шуме.

Раздел состоит из трех вкладок:

Вкладка	Что показывает	Когда смотреть
Тревоги	Открытые события, которые требуют внимания: критичные изменения прав, MFA, ролей, подозрительные файлы, массовые удаления, ночные входы, повторные неудачные попытки	В первую очередь после Telegram-сигнала или утром при проверке системы
События	Подробный журнал: кто, когда, что сделал, зона, результат, тип события	Когда нужно восстановить картину: входы, отказы, действия с заявками, файлами, план-графиком и системные ошибки
Отчёты	Ежедневные отчеты мониторинга: дата, период, статус отправки и краткая сводка	Для спокойной регулярной проверки, что система не молчала ночью

Важно понимать разницу между событием и тревогой. Событие - это запись факта: например, пользователь вошел, заявка изменена, файл скачан, сервер отдал ошибку. Тревога - это событие или серия событий, которые похожи на риск и требуют взгляда администратора. Обычный вход или обычное скачивание файла не должны превращаться в тревогу. А смена роли, изменение MFA, права план-графика, подозрительный файл или массовое удаление - должны.

Foreman не создает новую тревогу на каждый одинаковый повтор. Если повторяется тот же риск, система увеличивает счетчик и обновляет время последнего появления. Telegram тоже не должен превращаться в сирену без пауз: для **critical** действует mute-окно 30 минут, для **warning** - 60 минут. Это сохраняет главное: администратор видит сигнал быстро, но не тонет в одинаковых сообщениях.

Ежедневный отчет уходит в Telegram один раз в сутки за предыдущий московский календарный день. Даже если событий не было, короткий "тихий" отчет полезен: он подтверждает, что контур жив и что отсутствие тревог - это результат проверки, а не молчание сломанной системы.

Закрытие тревоги - это административная отметка "проверено". Лучше писать короткий комментарий: **нормальный ночной вход**, **смена роли согласована**, **ложное срабатывание после теста**, **доступ отключен по заявке руководителя**. Через несколько месяцев такая строка экономит больше времени, чем кажется.

1.15.7 Как создать пользователя аккуратно

1. Открыть `Пользователи`. 2. Нажать создание пользователя. 3. Заполнить ФИО. 4. Указать логин. 5. Указать должность человеческим языком. 6. Выбрать роль. 7. Задать или выдать пароль. 8. Проверить вход.

Должность лучше писать уважительно:

Плохо	Хорошо
<code>прораб</code> , если человек зам. директора	<code>зам. директора</code>
пусто	<code>сотрудник</code>
случайное старое значение	фактическая должность

Раздел 2. Мобильная версия Foreman

Мобильное приложение нужно для работы на объекте и с телефона.

Текущая версия в исходниках и свежем APK-релизе: 0.2.114+6115 .

2.1 Главный экран

На главном экране доступны карточки:

Раздел	Кому виден	Назначение
Мои реквизиты	Всем	Проверить свои данные
Пользователи	Админу	Управление пользователями
Модерация словарей	Админу	Проверка/правка словарей
Заявка на заполнения	Рабочим пользователям	Создание заявки на заполнения
Заявка на расходники	Рабочим пользователям	Создание заявки на расходники
Балконный/оконный блок	Рабочим пользователям	Заявка по оконно-балконному блоку
Переделка рамы/створки	Рабочим пользователям	Заявка на переделку
Журнал заявок	Рабочим пользователям	Просмотр заявок
Шаблоны замеров	Всем, но временно недоступен	Кнопка есть, но неактивна с подписью Ведутся профилактические работы
Сервер и связь	Всем	Проверка сервера и настроек

Что важно понимать:

- Мои реквизиты нужны не "для галочки": эти данные подставляются в заявки и помогают не набивать одно и то же руками;
- Пользователи в мобильном приложении доступны администратору: можно видеть запросы доступа, создавать пользователей, менять данные и пароль;
- Модерация словарей нужна администратору для контроля новых значений, которые появляются в заявках: формулы, примечания, расходники;
- Шаблоны замеров показаны как раздел, но временно недоступны для обычной эксплуатации: кнопка на главной неактивна, хотя экранные маршруты и серверные API уже предусмотрены.

2.2 Вход

Порядок:

1. Открыть приложение. 2. Ввести логин и пароль. 3. Нажать вход. 4. Если логин или пароль пустые, приложение покажет `Введите логин и пароль`. 5. Если сервер отклонил вход, будет сообщение об ошибке.

Дополнительные действия на экране входа:

Кнопка	Что делает
Восстановить доступ	Запросить восстановление
Активировать доступ	Активировать доступ по коду
Настройки сервера	Проверить или изменить сервер

Обычному пользователю не нужно менять сервер без администратора.

Доверенное мобильное устройство

В актуальной версии Foreman Mobile при входе создает постоянный `installationId` конкретной установки приложения и хранит его в защищенном хранилище телефона. Это не IMEI, не номер телефона и не Android ID. Это случайный идентификатор, созданный самим Foreman Mobile.

При входе приложение отправляет на сервер:

- `deviceId` - этот постоянный `installationId`;
- безопасный `deviceProfile` - Android, модель, версия приложения и другие минимальные технические признаки без опасных разрешений.

Backend не хранит сырой `deviceId`. Он сохраняет только SHA-256 hash экземпляра приложения и привязывает мобильную сессию к этому hash.

Зачем это нужно:

- если сотрудник потерял смартфон, администратор может заблокировать именно доступ Foreman Mobile на этом смартфоне;
- сам сотрудник при этом не блокируется полностью и может войти с нового устройства;
- активные серверные сессии Foreman Mobile с потерянного смартфона отзываются;
- заблокированный экземпляр Foreman Mobile больше не может повторно войти.

Важно понимать границу функции: это не удаленное стирание телефона. Если потерянный смартфон офлайн и разблокирован, локальные данные могут оставаться на устройстве до следующего контакта с сервером. Эта мера блокирует доступ к backend и отзывает серверные сессии.

Защита от явного анализа приложения

В Foreman Mobile добавлен первый anti-tamper контур. Его задача не в том, чтобы превратить обычный строительный сервис в неприступный банковский сейф, а в том, чтобы приложение не молчало, когда его запускают в явно подозрительной среде.

Приложение перед входом и во время активной сессии передает на сервер мягкие сигналы безопасности:

- запущено ли приложение как debug/tampered build;
- подключен ли отладчик или внешний tracer;
- есть ли признаки Frida/динамического анализа;
- есть ли признаки root на устройстве.

Политика сделана осторожной. Root сам по себе не блокирует сотрудника автоматически: приложение предупреждает и записывает риск-сигнал, потому что на пользовательских телефонах бывают спорные случаи. А вот debugger, TracerPid, Frida и debug/tampered build считаются уже не бытовой странностью, а явной попыткой анализа. В таком случае мобильный вход блокируется, текущая мобильная сессия очищается, а сотрудник видит понятное сообщение, что доступ к Foreman Mobile заблокирован и нужно обратиться к администратору.

Эта проверка не сканирует все приложения на телефоне, не требует IMEI, Android ID, номера телефона и не добавляет опасных Android-разрешений.

Для чтения

2.3 Восстановить доступ

Используется, если человек забыл пароль.

Типовые проверки:

- нужно ввести e-mail;
- затем логин, код и новый пароль;
- пароль должен быть не короче 6 символов;
- пароли должны совпадать.

После успеха приложение сообщает: **Пароль обновлен. Войдите в Foreman с новым паролем.**

Для чтения

2.4 Активировать доступ

Используется, когда администратор выдал код активации.

Порядок:

1. Ввести ФИО и телефон. 2. Получить/ввести код. 3. Ввести логин и пароль. 4. Подтвердить.

После успеха приложение сообщает: **Доступ активирован. Добро пожаловать в Foreman.**

2.5 Обновление приложения

Приложение проверяет обновления через подписанный манифест. Если обновление есть и подпись манифеста корректна, появляется предложение [Скачать и установить](#).

Перед установкой Foreman Mobile сначала проверяет подпись `app-update.json.sig`, затем скачивает APK через штатный Android DownloadManager в обычную папку «Загрузки» и сверяет контрольную сумму SHA-256 именно этого скачанного файла с подписанным манифестом обновления. Это нужно не для усложнения жизни пользователю, а чтобы поврежденный или подмененный файл не устанавливался как нормальная версия приложения.

После проверки Foreman передает этот же файл штатному Android-установщику. Если Android еще не разрешил Foreman устанавливать APK из этого источника, приложение открывает системную настройку именно для Foreman; после возврата установка продолжается через системное окно Android. Это по-прежнему не тихая установка: Android показывает пользователю подтверждение.

Порядок:

1. Нажать [Скачать и установить](#). 2. Дождаться загрузки. 3. Разрешить установку APK, если Android спросит. 4. Открыть обновленное приложение.

Если есть несинхронизированные черновики, не удалять приложение без консультации.

2.6 Заявка на заполнения

Открытие: [Главная](#) -> [Заявка на заполнения](#).

Ключевые кнопки:

Кнопка	Что делает
Создать/сохранить заявку	Создает или сохраняет черновик
Удалить заявку	Удаляет заявку, если разрешено
Предыдущие заявки	Показывает историю/предыдущие
Отправить на e-mail	Отправляет готовую заявку
Камера	Добавить фото с камеры
Галерея	Добавить фото из галереи
Подставить из профиля	Заполнить данные из профиля

Порядок:

1. Открыть форму. 2. Указать объект и номер заявки. 3. Заполнить строки. 4. Добавить фото при необходимости. 5. Нажать [Создать/сохранить заявку](#). 6. Проверить готовность. 7. Нажать [Отправить на e-mail](#).

Типовые сообщения:

Сообщение	Что означает
Укажите объект и номер заявки	Не заполнены обязательные поля
Сначала создай заявку	Нужно сначала сохранить черновик
Черновик восстановлен с телефона	Приложение подняло локальный черновик

2.7 Заявка на расходники

Открытие: Главная -> Заявка на расходники .

Порядок похож:

1. Указать объект. 2. Указать номер заявки. 3. Добавить строки материалов. 4. Указать количество. 5. Добавить примечание. 6. Сохранить. 7. Отправить на e-mail.

Типовые сообщения:

Сообщение	Что делать
Укажите объект и номер заявки	Заполнить обязательные поля
Сначала создай заявку	Сначала сохранить черновик
Создана новая версия: ...	Система создала новую версию заявки
Ошибка отправки: ...	Проверить связь, обязательные поля, e-mail

2.8 Балконный/оконный блок

Открытие: Главная -> Балконный/оконный блок .

Возможные варианты конструкций:

Вариант	Смысл
Один блок без обязательных импостов	Простая конструкция
Один блок, вертикальный импост обязателен	Нужен вертикальный импост
Один блок, горизонтальный импост обязателен	Нужен горизонтальный импост
Один блок в составе конструкции	Блок как часть конструкции
Два блока, второй блок справа	Балконный вариант из двух блоков

Рабочий порядок:

1. Выбрать тип конструкции. 2. Заполнить размеры. 3. Выбрать створки и импосты. 4. Проверить расчет. 5. Нажать [Перейти к отправке](#) . 6. При необходимости сохранить контакт. 7. Отправить.

Полезные кнопки:

- [Повторить расчет](#) ;
- [Одна на все](#) ;
- [По полям](#) ;
- [Копировать](#) ;
- [Применить ко всем полям](#) ;
- [Скопировать в остальные](#) ;
- [Открыть карточку](#) ;
- [Внести изменения](#) .

2.9 Переделка рамы/створки

Открытие: Главная -> Переделка рамы/створки .

Порядок:

1. Заполнить данные по переделке. 2. Указать размеры. 3. Проверить эскиз/предпросмотр. 4. Сформировать PDF, если требуется. 5. Отправить на e-mail.

Типовые сообщения:

Сообщение	Что означает
Заполните размеры для быстрого эскиза	Недостаточно данных для эскиза
Укажите e-mail контакта	Нужно ввести адрес
Некорректный e-mail	Исправить адрес
Контакт сохранён	Адресат сохранен

2.10 Журнал заявок в мобильном

Открытие: Главная -> Журнал заявок .

Используется для:

- просмотра списка заявок;
- открытия деталей;
- проверки статусов;
- перехода к доступным действиям.

В журнале есть быстрые фильтры:

Фильтр	Что показывает
Мои	Заявки текущего пользователя
Все	Общий список доступных заявок
Чужие	Заявки других пользователей, если право просмотра доступно
Все типы	Без ограничения по типу
Заполнения	Только заявки на заполнения
Расходники	Только заявки на расходники
Оконный блок	Только оконно-балконные заявки
Переделка	Только переделки рамы/створки

В карточке заявки мобильное приложение показывает объект, тип, стадию, автора, источник, версию, количество файлов и внешние вложения, если они есть. Доступные кнопки зависят от прав и статуса:

Открыть для редактирования , Внести изменения , Удалить заявку , отправка себе на e-mail.

Для массовой работы с документами, сортировками, таймлайном и гостевым просмотром лучше использовать веб.

2.11 Черновики и связь

Мобильное приложение содержит локальную логику черновиков и синхронизации.

Что это значит:

- если связь слабая, приложение старается сохранить работу;
- черновик может восстановиться с телефона;
- после появления связи нужно проверить, что заявка синхронизировалась;
- если приложение просит сначала создать заявку, нужно сохранить черновик перед отправкой.

2.12 Сервер и связь

Раздел нужен для проверки подключения.

Обычному пользователю:

- не менять сервер без администратора;
- при проблеме сделать скриншот;
- сообщить, с какого телефона и под каким логином входил.

Администратору:

- проверить URL сервера;
- проверить доступность API;
- проверить обновления;
- не удалять приложение, если есть локальные черновики.

2.13 Чего в мобильной версии нет

Чтобы не вводить пользователя в заблуждение:

- мобильное приложение не является заменой web-документохранилища;
- в мобильной версии нет полноценного раздела Документохранилище с папками, сортировками и массовыми файловыми действиями;
- в мобильной версии нет отдельного экрана Таймлайн ;
- в мобильной версии нет полноценного интерактивного План-графика ;
- гостевой наблюдатель и общий просмотр удобнее и правильнее использовать через Foreman Web;
- большие файловые операции, массовая отправка документов, скачивание PDF-отчета таймлайна и командная работа с план-графиком выполняются в браузере.

Мобильное приложение сильнее всего там, где человек находится на объекте: быстро создать заявку, сохранить черновик, добавить фото, открыть журнал и отправить заявку дальше.

Читать всем

Раздел 3. Telegram-бот

Этот раздел встроен в единое руководство вместо отдельного пересказа README. Пользователь должен открыть раздел `Telegram-бот` и получить всю рабочую информацию здесь: роли, меню, команды, права, сценарии, ограничения и типовые ошибки.

Telegram-бот нужен для быстрых действий в Telegram: найти заявку, найти файл, открыть личный кабинет, воспользоваться кнопками, отправить файл, работать с карточкой, запустить отгрузку, использовать голос, проверить служебные статусы администратору.

Важно: бот не заменяет веб и мобильное приложение. Сложное заполнение заявки удобнее делать в вебе или мобильном Foreman. Бот силен там, где нужна быстрая команда, привычный чат, личный кабинет или поиск.

Читать всем

3.1 Роли и права

Кто есть кто

Роль в Telegram-боте	Как определяется
Администратор	Telegram <code>user_id</code> входит в <code>TELEGRAM_ADMINS</code>
Ответственный пользователь	Telegram <code>user_id</code> привязан к ФИО в <code>TELEGRAM_USER_MAP</code> , и это ФИО совпадает с составителем заявки
Участник без привязки	Нет пары <code>Telegram user_id -> ФИО</code>

Что разрешено

Действие	Админ	Ответственный пользователь	Чужая заявка / без привязки
Просмотр карточки заявки	Да	Да	Да
Просмотр файлов из карточки	Да	Да	По <code>FILE_VIEW_STRICT_ACL</code>
Прямая команда <code>найди файл ... / пришли файл ...</code>	Да	Да	Да в текущем поведении
Правка формы заявки	Да	Только своей	Нет
Удаление файлов заявки	Да	Только своих	Нет
Dry-run удаления	Да	Только своих	Нет
Пересылка на e-mail	Да	Только своих	Нет
Dry-run отправки	Да	Только своих	Нет
Отгрузка на объект	Да	Только своих	Нет

Действие	Админ	Ответственный пользователь	Чужая заявка / без привязки
Восстановление удаленных файлов	Только назначенные администраторы	Нет	Нет
Админ-отчеты, метрики, бэкапы	Да	Нет	Нет

Прямое правило: пользователи не могут удалять чужие заявки и чужие файлы. При попытке бот отвечает: `Удаление запрещено: можно удалять только свои файлы.`

Важное про восстановление

Восстановление удаленных файлов разрешено только назначенным администраторам. Если обычный пользователь пытается восстановить файл, бот должен направить его к администратору.

Для чтения

3.2 Меню, помощь и кнопки

Команды открытия меню

Бот понимает:

- `справка` ;
- `меню` ;
- `команды` ;
- `помощь` ;
- `подсказка` ;
- `что умеешь` ;
- `глобальное меню` ;
- `справочное меню` ;
- `покажи справку` ;
- `покажи меню` ;
- `покажи список команд` ;
- `help` ;
- `/help` ;
- `/menu` .

Мини-сценарий:

1. Пользователь пишет `меню` . 2. Бот отвечает коротко: `Выберите раздел кнопками` . 3. Показывает корневые кнопки. 4. В группе бот держит отдельное служебное меню, чтобы рабочие уведомления не смешивались с навигацией.

Корневые разделы бота

Раздел	Что делает
<code>ПОИСК ЗАЯВОК</code>	Поиск по номеру заявки, объекту, ФИО, заказу, префиксу
<code>ТАБЛИЦА УЧЕТА</code>	Текстовый или графический срез учетной таблицы

Раздел	Что делает
СОЗДАНИЕ ЗАЯВОК	Запуск конструкторов заявок
ПРАВКА И ОТПРАВКА	Правка, e-mail, действия из карточки
ОТГРУЗКА	Запросы отгрузки на объект
УДАЛЕНИЕ И ВОССТАНОВЛЕНИЕ	Безопасное удаление, dry-run, восстановление
ЖУРНАЛ	Журнал действий по заявке
РЕЕСТР ЗАПУСКОВ	Поиск по запуску, объекту, номеру заказа
ГОЛОС	Голосовые команды и диктовка
ШАБЛОНЫ ДЛЯ ЗАМЕРОВ	Шаблоны замеров, WebApp, PDF/PNG
ВЫХОД	Выход из текущего сценария

Если пропали кнопки

Команды:

- кнопки ;
- покажи кнопки ;
- обнови кнопки ;
- верни кнопки .

Что делает бот:

- восстанавливает нижнюю клавиатуру;
- в группе пересобирает служебное меню;
- в личке снова показывает корневые кнопки;
- не требует перезапуска всего диалога.

Для чтения

3.3 Личный и общий кабинет

В общем чате нижняя клавиатура:

- Меню ;
- Личный кабинет .

В личном кабинете:

- Меню ;
- Общий кабинет .

Что важно:

- личный кабинет - отдельный приватный чат с ботом;
- действия в личке не публикуются в общий чат автоматически;
- в личке удобно смотреть свои заявки и свои последние файлы;
- в общий кабинет возвращаются через кнопку или команду.

Команды перехода:

- личный кабинет ;
- войти в личный кабинет ;
- открой личный кабинет ;
- перейди в личный кабинет ;
- общий кабинет ;
- войти в общий кабинет ;
- открой общий кабинет ;
- открой меню .

Полезные личные команды:

- мои заявки ;
- мои заявки 10 ;
- мои заявки 15 ;
- мои последние заявки ;
- мои свежие заявки ;
- мои недавние заявки ;
- мои заявки за последнее время ;
- мои последние файлы ;
- мои последние файлы 10 ;
- мои последние документы ;
- мои последние вложения ;
- мои последние эскизы .

Если у пользователя нет привязки в `TELEGRAM_USER_MAP` , персональные команды могут не сработать. Нужно обратиться к администратору.

Для чтения

3.4 Поиск заявок

Поиск одной заявки

Форматы:

- ЖА-34 ;
- жа34 ;
- покажи ЖА-34 ;
- действия по заявке ЖА-34 ;
- что можно сделать по заявке ЖА-34 .

Что делает бот:

1. Нормализует номер.
2. Ищет заявку.
3. Показывает карточку: составитель, объект, заказ, дата, статус.
4. Предлагает доступные действия.

Поиск по номеру заказа

Форматы:

- С-26-00314 ;

- C26-00314 ;
- C2600314 ;
- C-148 ;
- C148 ;
- C 148 ;
- 00314 .

Что делает бот:

- очищает номер от лишних символов;
- ищет точное совпадение или совпадение по хвосту номера;
- показывает заявку, связанную с заказом.

Поиск по объекту

Форматы:

- заявки по прайм ;
- заявки по объекту прайм ;
- покажи заявки по прайму ;
- найди заявки по прайму ;
- покажи заявки по объекту сенат ;
- заявки по прайму .

Как работает:

- бот ищет в названии объекта;
- понимает склонения: прайму -> прайм , новосаратовке -> новосаратовк ;
- показывает нумерованный список;
- после списка можно ввести номер строки, чтобы открыть полную карточку.

Поиск по ФИО составителя

Форматы:

- найди заявки от жалялова ;
- покажи заявки жалялова ;
- выведи заявки жалялова ;
- заявки от П.И. Жалялова ;
- по составителю Жалялов ;
- покажи заявки по составителю Жалялов .

Если фамилия неоднозначная, бот должен показать варианты по людям, а не угадывать.

Поиск по префиксу

Форматы:

- покажи заявки ЖА ;
- покажи 5 заявок ЖА ;
- покажи жа ;

- `найди заявки СА` .

Что делает бот:

- выбирает заявки по префиксу;
- сортирует свежие сверху;
- показывает ограниченный список;
- позволяет открыть карточку по номеру строки.

Раздел `поиск заявок` кнопками

Кнопки раздела:

- `ОДНА ЗАЯВКА` ;
- `ГРУППА ЗАЯВОК` ;
- `ПО НОМЕРУ ЗАКАЗА` ;
- `ПО ОБЪЕКТУ` ;
- `ПО ФИО` ;
- `МОИ ЗАЯВКИ` ;
- `МОИ ФАЙЛЫ` ;
- `ЕЩЁ ФИЛЬТРЫ` .

Кнопка `ЕЩЁ ФИЛЬТРЫ` ведет к рабочим фильтрам:

- `НЕЗАПУЩЕННЫЕ` ;
- `НЕ ЗАКАЗАННЫЕ` ;
- `НА БАЗЕ` .

Для чтения

3.5 Таблица учета

Текстовая таблица

Команды:

- `покажи таблицу` ;
- `покажи таблицу на 10` ;
- `покажи в таблице последние 15` .

Что делает бот:

- показывает последние заявки текстом;
- по умолчанию может показать около 20 строк;
- ограничивает длинные списки, чтобы не забивать чат.

Красивая таблица

Команды:

- `покажи таблицу красиво` ;
- `покажи таблицу красиво на 10` .

Что делает бот:

- формирует PNG-картинку с таблицей;
- использует цветовую подсветку;
- при ошибке рендера не должен падать, а должен ответить понятным текстом.

Для чтения

3.6 Файлы заявок

Базовый поиск файлов

Команды:

- покажи файл ЖА-34 ;
- покажи файлы ЖА-34 ;
- пришли файл ЖА-34 ;
- пришли файлы ЖА-34 ;
- пришли мне файл ЖА-34 ;
- пришли файл сюда ЖА-34 ;
- найти файл ЖА-34 ;
- найти файлы ЖА-34 ;
- найди файл ЖА-34 ;
- найди файлы ЖА-34 .

Что делает бот:

- если файл один, отправляет его;
- если файлов несколько, показывает выбор;
- если версий несколько, предлагает выбрать нужную;
- может работать по префиксу, если номер помнится не полностью.

Последние персональные файлы

Команды:

- мои последние файлы ;
- мои последние файлы 10 ;
- мои последние документы ;
- мои последние вложения ;
- мои последние эскизы ;
- мои файлы за последнее время .

Что показывает бот:

1. ЖА-34 - эскиз ПВХ-блока - 12.03.2026 (создан в боте)
2. ЖА-35 - расходники - 12.03.2026 (получен по e-mail)

После списка можно нажать номер файла или отправить 0 для выхода.

Безымянные файлы

Команды:

- `покажи безымянные ;`
- `покажи безымянные файлы ;`
- `покажи файлы без имени .`

Для неадминов бот должен фильтровать результат по владельцу, чтобы чужие файлы не утекали.

Читать всем

Защита входящих файлов

Бот проверяет файлы, которые приходят в чат или по e-mail. Важно понимать точнее: речь про письма с файлами на почту чат-бота, которые бот пытается сохранить на сервере и связать с рабочим процессом. До сохранения они проходят защитную проверку.

- расширение входит в белый список `ALLOWED_ATTACHMENT_EXTENSIONS` ;
- размер не превышает `MAX_ATTACHMENT_MB` , обычно 45 МБ;
- файл не похож на опасный исполняемый файл;
- сигнатура соответствует типу.
- файл проходит внешний антивирусный API, если этот контур включен на сервере.

Если файл подозрительный, бот отклоняет его с понятной причиной и не ломает диалог.

Для спокойной проверки есть несколько команд:

Команда	Что делает
<code>покажи антивирус</code>	Показывает последние проверки файлов: имя, источник, итог проверки и время
<code>покажи проверки файлов</code>	Делает то же самое, если сотрудник помнит старую формулировку
<code>покажи заблокированные файлы</code>	Показывает безопасный список файлов, которые не попали в обычную работу
<code>покажи безопасность</code>	Для администратора: показывает свежие события безопасности Foreman - входы, отказы гостю, лимиты скачивания и скачивания файлов

Эти команды не показывают содержимое файлов, пароли или токены. Они нужны, чтобы сотрудник не нервничал: система не молчит, а оставляет понятный след проверки. Команда `покажи безопасность` доступна только администратору, потому что в ней видны служебные события входа и отказов.

Читать всем

3.7 Карточка заявки в Telegram

Открыть карточку можно просто номером заявки, например:

ЖА-34

Типовое меню карточки:

Номер	Действие
1	Просмотреть файлы
2	Отредактировать заявку
3	Отправить на e-mail
4	Удалить
5	Отредактировать выбранную версию
6	Опубликовать в общем кабинете
0	Оставить без изменений

Что важно:

- карточка хранит контекст текущей заявки;
- не нужно каждый раз заново вводить номер;
- выбор файлов может поддерживать диапазоны вроде 1-3 ;
- SVG-эскизы бот старается отправлять как PNG, чтобы было удобнее на смартфоне;
- если действие запрещено правами, бот должен отказать.

Кто считается хозяином заявки

Бот различает номер заявки и право на живую правку.

Ситуация	Как бот должен действовать
Своя форма, свой номер	Обычный сценарий, автор правит сам
Своя форма, чужой префикс	Право правки остается за фактическим автором формы; владельцу номера может уйти уведомление
Внешний файл под чужим номером	Редактировать могут загрузивший файл и владелец номера по Sheets
Старый архив без автора	Мягкий режим совместимости, без жестких новых привилегий

Для чтения

3.8 Правка заявок

Команды:

- правка ;
- правка ЖА-34 ;
- правка заявки ЖА-34 ;
- правка заполнения ЖА-355 ;
- правка расходники ЖА-888 .

Порядок:

1. Пользователь вводит команду. 2. Бот открывает сохраненную форму. 3. Ведет по полям редактирования. 4. После изменений пересобирает визуал и сохраняет новую версию.

Защита:

- чужую заявку править нельзя;
- если данные менялись параллельно, бот не должен молча затереть чужую работу;
- при спорной правке владелец номера может получить уведомление.

Читать всем

3.9 Создание заявок через Telegram-бота

Команды:

- `создай заявку ;`
- `создать заявку ;`
- `создание заявки ;`
- `хочу создать заявку ;`
- `хочу сделать заявку ;`
- `помоги создать заявку ;`
- `шаблон заявки ;`
- `бланк заявки ;`
- `нужен шаблон заявки ;`
- `нужен бланк заявки .`

Прямые входы:

- `создай заявку на заполнения ;`
- `создать заявку на заполнения ;`
- `создай балконный блок ;`
- `создать балконный блок ;`
- `создай окно ;`
- `создай заявку на расходники ;`
- `покажи расходники ;`
- `заявка расходники ;`
- `создай заявку на переделку рамы ;`
- `создай заявку на переделку створки ;`
- `заявка для альтавина: ... ;`
- `заявка в альтавин: ... .`

Когда использовать бот:

- если человек уже привык к командному конструктору;
- если удобнее надиктовать голосом;
- если нужна быстрая заявка в чате.

Когда лучше веб или мобильное приложение:

- много строк;
- много вложений;
- нужна аккуратная проверка формы;
- нужно работать с документохранилищем.

Удаление

Команды:

- удали файл ... ;
- удалить файл ... ;
- удали безымянные файлы ;
- удали файл без имени .

Правила:

- чужие файлы удалять нельзя;
- удаление должно иметь подтверждение;
- бот не должен удалять молча;
- пользователь может удалять только то, на что у него есть право.

Dry-run удаления

Команды:

- проверь удаление ... ;
- проверить удаление

Dry-run показывает, что будет удалено, но не удаляет реально. Это полезно перед рискованным действием.

Восстановление

Команды:

- восстанови файл ... ;
- восстановить файл ... ;
- восстанови заявку

Восстановление доступно только назначенным администраторам.

Команды:

- перешли файл ... ;
- перешли заявку ... ;
- вышли файл ... ;
- вышли заявку ... ;
- отправь на почту файл ... ;
- отправить на почту документ

Dry-run отправки:

- проверь отправку ... ;

- проверить отправку

Порядок:

1. Открыть карточку заявки или вызвать отправку командой. 2. Выбрать файл или заявку. 3. Выбрать адресата или ввести адрес. 4. Для ручного ввода адреса в меню можно использовать `руч`. 5.

Подтвердить отправку.

Отправка разрешена администратору или ответственному пользователю по своей заявке.

Для чтения

3.12 Отгрузка на объект

Команды запуска:

- `прошу отгрузить ... ;`
- `нужно отгрузить ... ;`
- `надо отгрузить ... ;`
- `отгрузить ... ;`
- `отгрузка на объект ;`
- `запрос отгрузки ;`
- `запросить отгрузку ;`
- `свяжись ... ;`
- `да свяжись ... ;`
- `организуй доставку ... ;`
- `организуй вывоз ... .`

Отмена:

- `я передумал ;`
- `передумал ;`
- `не надо ;`
- `стоп ;`
- `отмена ;`
- `отбой отгрузки ;`
- `отмена отгрузки ;`
- `отмени отгрузку .`

Обычно бот ведет по шагам: какая заявка, что отгрузить, когда, подтверждение.

По необходимости

3.13 Реестр запусков

Раздел используется для поиска по производственным данным.

Вход:

- `реестр запусков ;`
- `войти в реестр запусков ;`
- `запуск 4 ;`
- `зап.4 ;`

- что входит в запуск 10 ;
- сводка Универ ;
- история Универ .

Внутри раздела можно писать короче:

- 84 ;
- Универ ;
- С-26-00307 ;
- сводка Универ ;
- история Универ .

Явные команды вне раздела:

- реестр ... ;
- запуск ... ;
- зап ... ;
- что входит в запуск ... ;
- сводка ... ;
- история

Служебные команды:

- реестр статус ;
- статус реестра .

Выход:

- 0 ;
- назад ;
- отмена ;
- выход ;
- стоп ;
- закрыть .

По необходимости

3.14 Шаблоны для замеров

Вход:

- шаблоны для замеров ;
- меню замеров ;
- замеры ;
- замеры этажа ;
- шаблон замеров .

Основной путь - кнопка web App в Telegram. Она открывает интерфейс с объектами, активными листами, zoom/pan, review-модалкой и e-mail-модалкой.

Текстовый fallback сохранен для совместимости.

Массовый ввод по группе:

- ОК-1 2130 830 ;
- ОК1 2130x830 ;
- ОК 1 2130 830 ;
- ОК-1пр* 2130 830 .

Точечный ввод:

- 4 2145 845 ;
- 1,2 2200 800 ;
- 1 2 2200 800 ;
- RP1.2_с1.1_01_2_1 2145 845 .

Многоблочные конструкции:

- 7 2200 700 1400 800 ;
- 7 62 1400 820 .

Сервис внутри шаблона:

- список ;
- покажи список ;
- покажи шаблон ;
- сброс ;
- сформируй ;
- нн ;
- ∅ .

Результат:

- PDF;
- PNG;
- запись в мои последние файлы ;
- быстрый возврат к последней версии через мои последние файлы .

По необходимости

3.15 Голосовой режим

Голосом можно:

- открыть меню;
- войти в личный кабинет;
- открыть общий кабинет;
- запускать поддерживаемые сценарии;
- диктовать часть рабочих данных.

Команды:

- войти в личный кабинет ;
- открой личный кабинет ;
- перейди в личный кабинет ;
- войти в общий кабинет ;
- открой меню .

В голосовых сценариях `0` , `ноль` , `нуль` могут использоваться для выхода или возврата в поддерживаемых диалогах.

Если бот не понял голос, нужно повторить коротко: не длинной фразой на полминуты, а одной рабочей командой.

По необходимости

3.16 Многосоставные команды

Бот может понимать цепочки действий:

- `покажи ЖА-43 и перешли файл Иванову` ;
- `открой ЖА-43 и найди файлы и запросить отгрузку` ;
- `жа43 открой и перекинь файл Иванову` ;
- `найди ПЖ-31 и ЖА-43` ;
- `покажи ЖА-43 и найди файлы по ЖА-43 и покажи шаблон заявки на расходники` ;
- `покажи мои заявки, найди файлы по ЖА-43, запросить отгрузку ЖА-43` .

Практическое правило: если бот начал путаться, разбить команду на короткие шаги.

Для чтения

3.17 Личное общение

Команды входа:

- `личка` ;
- `приват` ;
- `приватное общение` ;
- `личное общение` .

Для чего нужно:

- передать файл коллеге напрямую;
- написать сообщение в личный кабинет адресата;
- не публиковать файл в общий чат.

Администратору

3.18 Админские команды

Админ - пользователь, чей `user_id` входит в `TELEGRAM_ADMINS` .

Команда или фраза	Кто может	Куда идет результат
<code>/status</code>	Админ	Диагностика в тот же чат
<code>/modcheck</code> , <code>modcheck</code> , <code>проверка фильтра</code> , <code>проверь фильтр</code>	Админ	Отчет фильтра в тот же чат
<code>покажи риски</code> , <code>покажи аномалии</code>	Админ	Ответ в тот же чат
<code>покажи метрики</code> , <code>метрики</code>	Админ	Ответ в тот же чат
<code>покажи индекс файлов</code> , <code>покажи file index</code>	Админ	Ответ в тот же чат

Команда или фраза	Кто может	Куда идет результат
<code>покажи бэкап</code> , <code>покажи backup</code>	Админ	Ответ в тот же чат
<code>сделай бэкап</code> , <code>сделай backup</code>	Админ	Архив на сервере и ответ в чат
<code>/larisa_report_now</code> , <code>larisa_report_now</code>	Админ	Полный отчет в основной рабочий чат, подтверждение админу
<code>/larisa_report_at</code> ЧЧ:ММ	Админ	Отчет в рабочий чат в заданное время
<code>/mail_pending_claims</code> , <code>покажи почтовые ожидания</code>	Админ	Показывает заявки, которые бот увидел во входящей почте, но не нашел в Google-таблице; рядом есть подсказка для исправления ошибочно распознанного номера
<code>/fix_mail_claim</code> <ошибочный номер> <правильный номер>	Админ	Переносит почтовую историю с ошибочного номера на правильный и убирает ошибочный номер из контроля незаведенных заявок
<code>/clear_mail_pending</code>	Админ	Справка по реестру почтовых ожидающих
<code>/clear_mail_pending</code> <номер>	Админ	Удаляет одну заявку из почтового реестра ожиданий в PostgreSQL
<code>/clear_mail_pending</code> subject_only	Админ	Удаляет записи уровня "номер только в теме письма"
<code>/clear_mail_pending</code> all	Админ	Очищает весь реестр ожидающих
<code>/email_history</code> , <code>/mail_history</code> , <code>покажи письма</code>	Админ	История отправленных e-mail
<code>/critical_mail_queue</code> , <code>/mail_queue</code> , <code>покажи очередь писем</code>	Админ	Очередь важных писем: подтверждения прорабам, письма Оксане/Ларисе/руководству, ALARM по вирусам, retry/dead/sent без текста писем и секретов
<code>еженедельник</code> , <code>/damage_forecast</code> , <code>/weekly_control</code>	Админ	Живой прогноз на ближайшие 72 часа: где скоро истечет контрольный срок, кому бот может написать и где может потребоваться руководство
<code>/mail_manual_review</code> , <code>покажи непонятные письма</code> , <code>покажи разбор писем</code>	Админ	Письма, которые бот не смог уверенно разобрать сам и передал человеку на ручной просмотр
<code>/file_security_history</code> , <code>/file_security</code> , <code>/antivirus</code> , <code>покажи антивирус</code> , <code>покажи проверки файлов</code>	Админ	Последние проверки файлов без содержимого файлов и без секретов
<code>/file_security_quarantine</code> , <code>/blocked_files</code> , <code>покажи заблокированные файлы</code>	Админ	Безопасный список файлов, которые система не пустила в обычную работу
<code>/access_security</code> , <code>/security_audit</code> , <code>покажи безопасность</code>	Админ	Журнал входов, отказов гостю, лимитов скачивания и скачиваний файлов без паролей, токенов и содержимого файлов

Администратору Вечерний контроль и почтовые ожидания

В Telegram-боте есть служебный контур вечернего контроля: он помогает увидеть, какие заявки попали в рабочий поток, какие пришли по почте, где есть "ожидающие" записи и что требует внимания администратора.

Что важно пользователю:

- обычному сотруднику не нужно вручную трогать служебные JSON-файлы;

- администратор может вручную запустить отчет командой `/larisa_report_now`;
- администратор может временно поменять время отчета командой `/larisa_report_at ЧЧ:ММ`;
- администратор может посмотреть именно почтовые ожидания командой `/mail_pending_claims` или фразой `покажи почтовые ожидания`;
- если прораб ошибся в теме письма или имени файла, администратор исправляет связь командой `/fix_mail_claim <ошибочный номер> <правильный номер>`;
- реестр почтовых “ожидających” чистится только осознанно через `/clear_mail_pending ...`;
- история уже отправленных e-mail смотрится через `/email_history`, `/mail_history` или `покажи письма`;
- очередь важных писем смотрится отдельно: `/critical_mail_queue`, `/mail_queue` или `покажи очередь писем`;
- прогноз контрольных событий на ближайшие 72 часа смотрится командой `еженедельник`;
- непонятные ответы Оксаны/Ларисы смотрятся отдельно: `/mail_manual_review`, `покажи непонятные письма` или `покажи разбор писем`.

Практический смысл: бот не просто отвечает на команды, а помогает не потерять заявки и письма, которые уже прошли через рабочий контур.

Важно различать две вещи. `покажи письма` - это история того, что уже ушло через SMTP и попало в журнал отправленных писем. `покажи очередь писем` - это рабочая очередь: там видно, что еще ждет отправки, что будет повторяться, что уже отправлено, а что умерло после повторных попыток и требует внимания администратора. Полный текст писем, пароли, токены и вложения в этой очереди не показываются.

Команда `еженедельник` показывает не историю и не очередь уже созданных писем, а живой прогноз на ближайшие 72 часа. Бот заново смотрит текущие данные: какие заявки еще не найдены в Google-таблице, что уже было отправлено Оксане или Ларисе, где скоро истечет контрольный срок и где может потребоваться руководство. Если заявка появилась в Google-таблице, она исчезнет из прогноза при следующем запросе. Команда ничего не отправляет и ничего не меняет, это только диспетчерская сводка для администратора.

Новые служебные состояния почтового контроля хранятся не в случайных файлах, а в PostgreSQL: реестр контроля заявок, счетчики повторов, обработанные события, очередь важных писем и этапы обработки входящих писем. Это важно для Amvera: если контейнер перезапустился во время деплоя или почтовый сервер временно не ответил, состояние не должно исчезнуть вместе с процессом.

Ответ прорабу после получения файлов

Когда бот получает письмо с файлами заявки, он отправляет отправителю подтверждение: файлы получены, антивирусная проверка выполнена, дальше бот проконтролирует запуск заявки в работу.

В подтверждение добавлен компактный мини-таймлайн по ранее присланным письмам с этого e-mail, максимум `10` строк. Он показывает не “личные заявки прораба по фамилии”, а именно последние активные заявки, которые проходили через этот почтовый ящик. Это важно для общих рабочих почт: один сотрудник может временно отправлять заявки за другого прораба.

В мини-таймлайн не попадают заявки, которые уже отгружены на объект. Новая только что полученная заявка показывается как новая и не получает ложный статус из старой Google-таблицы.

В справочном блоке письма бот показывает доступ к Foreman Web. Если по e-mail найден персональный логин сотрудника, показывается только персональный доступ. Гостевой логин в таком письме не дублируется. Если персонального доступа нет, бот показывает гостевой вход только для просмотра.

Контроль заявки до внесения в Google-таблицу

Этот сценарий относится к заявкам по ущербу, которые пришли боту по e-mail с файлом, но еще не появились в Google-таблице `Учет заявок по ущербу`.

Главное правило: первые сутки бот никому не пишет по цепочке контроля. В интервале `0-24ч` он только проверяет, появилась ли строка в Google-таблице. Если строка появилась, контроль закрывается без писем Оксане, Ларисе и руководителю.

Если через 24 часа строки нет, бот запускает спокойную рабочую цепочку:

Этап	Что делает бот	Зачем
1	Пишет Оксане Владимировне	Уточнить, заведена ли заявка в Альтавин и есть ли номер запуска
2	При корректном номере запуска пишет Ларисе Разиповне	Попросить внести заявку в Google-таблицу и поставить актуальный статус
3	Если цепочка не закрылась, подключает руководителя	Показать, где заявка зависла, и попросить помочь запустить ее в работу

Лариса подключается только после корректного ответа Оксаны с номером запуска вида `C-26-00677`. В первые 24 часа писем Ларисе быть не должно.

В письмах по этому сценарию бот добавляет короткую хронологию и компактную схему задержки. Это нужно не для технических подробностей, а чтобы сотрудник сразу видел: когда заявка пришла, кто уже был подключен и почему контроль еще не закрыт.

Сценарий считается завершенным, когда заявка появляется в Google-таблице `Учет заявок по ущербу`.

Если пришло письмо от бота по незаведенной заявке, лучше отвечать коротко и в понятном формате:

ЗАВЕДЕНА C-26-00677

или:

НЕ ЗАВЕДЕНА

Свободный длинный ответ может попасть в ручной разбор, потому что бот не должен угадывать спорный смысл переписки.

Администратору `/status`

Показывает диагностику:

- `RELEASE ;`
- `SHEETS ;`

- `MAIL` ;
- `MAIL_SPOOL` ;
- `DEAD_LETTER_SIZE` ;
- `file_security` - включена ли проверка файлов, настроен ли внешний антивирусный контур и есть ли последние события;
- `critical_mail_queue` - очередь важных писем в PostgreSQL: подтверждения, письма Оксана/Ларисе/руководству и ALARM по вирусам;
- `critical_mail_sent_unconfirmed` - предохранитель на случай, когда SMTP уже отправил письмо, а запись `sent` в базе временно не подтвердилась;
- `mail_processing` - этапы обработки входящих писем: нет ли зависшего состояния после сохранения файлов;
- `HEARTBEAT_AGE` ;
- при включенном контуре - состояние реестра запусков.

Для внешней HTTP-проверки контейнера надежнее использовать `GET /healthz` и `GET /readyz` .

Администратору `/modcheck`

Проверяет фильтр мата:

- базовые формы;
- смешанный алфавит;
- leet-замены;
- мягкий мат;
- белый список.

Обычным пользователям команда не нужна.

История e-mail

Команды:

- `/email_history` ;
- `/mail_history` ;
- `покажи письма` .

Что видно:

- дата и время отправки;
- кому предназначалось письмо;
- e-mail адрес;
- тема письма;
- по номеру строки - полный текст письма и связанная цепочка общения, если она есть.

Журналы:

- `mail_history.jsonl` ;
- `chat_dialog_history.jsonl` .

Администратору Очередь важных писем

Команды:

- `/critical_mail_queue ;`
- `/mail_queue ;`
- `покажи очередь писем .`

Эта команда не заменяет `покажи письма` . Она показывает не историю, а рабочее состояние доставки важных писем:

- подтверждения прорабам;
- письма Оксане, Ларисе и руководству;
- ALARM-предупреждения по вирусам;
- письма в статусе `queued` , `retry` , `sent` , `dead` или `canceled` .

Если статус `dead` , значит письмо не доставлено после повторных попыток. Это уже не “подождать само пройдет”, а сигнал администратору: нужно проверить SMTP, адресата, очередь и при необходимости отправить письмо вручную.

Команда безопасная: она не показывает полный текст письма, вложения, пароли, токены и внутренние ключи очереди.

Администратору Почтовые ожидания

Команды:

- `/mail_pending_claims ;`
- `покажи почтовые ожидания .`

Это отдельная админская проверка для случаев, когда бот увидел заявку во входящей почте, но такой номер пока не найден в Google-таблице.

Команда помогает поймать человеческие ошибки до того, как они разрастутся в путаницу. Например: прораб написал в теме `как-58` , а по факту это `КАА-58` ; или имя файла похоже на один номер, а в Google-таблице заявка заведена под другим номером.

В ответе видно:

- сколько таких почтовых ожиданий сейчас есть;
- какой номер бот держит на контроле;
- как долго он ждет;
- тема письма;
- замаскированный адрес отправителя;
- какие форматы файлов найдены;
- подсказка для исправления.

Если номер распознан неправильно, используется команда:

```
/fix_mail_claim <ошибочный номер> <правильный номер>
```

Пример:

```
/fix_mail_claim ПУЛЬС-59 КАА-59
```

```
/fix_mail_claim IMG-20260527 КАА-58
```

Эта команда не удаляет историю письма. Она переносит почтовую историю на правильную заявку и убирает ошибочный номер из контроля незаведенных заявок. Это важно: после исправления бот не должен снова писать Оксане по старому неправильному номеру.

Для ручных связок действует дополнительная защита на HTTP-выдаче Foreman Web: ошибочный почтовый номер не должен появляться отдельной строкой в журнале и таймлайне, если он связан с правильной заявкой. Внешние вложения при этом учитываются у правильной заявки.

Администратору

Ручной разбор непонятных писем

Команды:

- `/mail_manual_review;`
- `покажи непонятные письма;`
- `покажи разбор писем.`

Сюда попадают ответы, которые бот не смог уверенно понять сам. Например: Оксана ответила без номера заявки, дала непонятный номер запуска или написала свободным текстом вместо короткого рабочего формата.

Задача этой команды - не ругать сотрудника и не спорить с письмом, а показать администратору: вот сообщение, которое требует человеческого взгляда. В ответе выводится только безопасная краткая информация: причина, отправитель, тема, найденные заявки и короткий фрагмент текста.

Справочно

3.19 Устойчивость к паузам и ошибкам

Бот дорабатывался под живую работу, где человек может:

- отвлечься на звонок;
- закрыть Telegram;
- вернуться позже;
- ошибиться в вводе;
- зависнуть в старом меню;
- потерять часть контекста из-за сбоя сети или рестарта сервиса.

Как бот должен вести себя:

- не тащить старый шаг в новый разговор;
- устаревший сценарий закрывать мягко;
- если состояние сохранено - поднимать пользователя с понятного шага;
- если контекст сомнительный - просить уточнение;
- не угадывать опасные действия.

Если бот отвечает не по теме:

1. Написать `выход`. 2. Написать `меню`. 3. Если пропали кнопки - написать `верни кнопки`. 4. Повторить короткую команду.

3.20 Короткий словарь рабочих команд

Задача	Команды
Открыть меню	меню , справка , /menu , /help , покажи меню
Вернуть кнопки	кнопки , покажи кнопки , обнови кнопки , верни кнопки
Личный кабинет	личный кабинет , открой личный кабинет
Общий кабинет	общий кабинет , открой общий кабинет
Одна заявка	ЖА-34 , покажи ЖА-34
По префиксу	покажи заявки ЖА , покажи 5 заявок ЖА
По заказу	С-26-00314 , С148 , С 148
По объекту	покажи заявки по прайму , заявки по объекту сенат
По ФИО	найди заявки от жалялова , по составителю Жалялов
Мои заявки	мои заявки , мои свежие заявки , мои заявки 10
Мои файлы	мои последние файлы , мои последние документы
Таблица	покажи таблицу , покажи таблицу красиво
Файлы заявки	пришли файл ЖА-34 , найди файлы ЖА-34
Правка	правка ЖА-34 , правка расходники ЖА-888
Создание	создай заявку , создай окно , создай заявку на расходники
Удаление	удали файл ... , проверь удаление ...
Восстановление	восстанови файл ... , восстанови заявку ...
E-mail	перешли файл ... , отправь на почту файл ...
Отгрузка	прошу отгрузить ... , запросить отгрузку
Реестр запусков	реестр запусков , запуск 4 , сводка Универ
Замеры	шаблоны для замеров , замеры этажа , сформируй
Голос	открой меню , войти в личный кабинет

3.21 Что осталось в отдельном README бота

Отдельный README Telegram-бота можно использовать как технический архив и подробную справку для разработчика. Обычному пользователю он не нужен: рабочая инструкция по боту включена в этот единый документ.

Практика

Приложение А. Сквозные рабочие сценарии

Для чтения

А.1 Прораб создает заявку на объекте

Лучший канал: мобильное приложение.

1. Открыть Foreman Mobile. 2. Выбрать тип заявки. 3. Заполнить объект и номер. 4. Добавить строки. 5. Добавить фото через **Камера** или **Галерея**. 6. Нажать **Создать/сохранить заявку**. 7. Проверить данные. 8. Нажать **Отправить на e-mail**. 9. Позже открыть веб и проверить заявку в журнале.

Для чтения

А.2 Менеджер проверяет заявку на ПК

Лучший канал: веб.

1. Открыть Foreman Web. 2. Открыть **Журнал**. 3. Найти заявку поиском или фильтром. 4. Открыть карточку. 5. Проверить вложения. 6. Скачать нужные файлы. 7. Проверить журнал доставки.

Для чтения

А.3 Прораб прикрепляет файл, который уже есть в Foreman

Лучший канал: веб.

1. Открыть заявку. 2. В **Вложения** нажать **Из документов**. 3. Выбрать папку. 4. Отметить файл. 5. Нажать **Прикрепить к заявке**. 6. Убедиться, что файл появился справа.

Главная польза: файл не нужно скачивать на ПК и загружать обратно.

Для чтения

А.4 Руководитель смотрит движение заявок

Лучший канал: веб.

1. Открыть **таймлайн**. 2. Проверить период **11 назад / 2 вперед**. 3. Нажать **обновить**. 4. Смотреть длинные цветные участки. 5. Нажать **скачать PDF**, если нужен отчет.

По необходимости

А.5 Команда работает с план-графиком

Лучший канал: веб.

1. Открыть **план-график**. 2. Найти нужный объект и дату. 3. Перетащить карточку внутри объекта, если нужно изменить плановую ячейку. 4. Поставить только тот статус, который относится к вашей зоне ответственности. 5. Нажать **обновить**, если нужно увидеть свежие изменения коллег. 6. Если нужен

отчет, выбрать диапазон дат, сформировать PDF и при необходимости отправить его на выбранные e-mail адреса. 7. Перед рассылкой проверить в окне, какие номера приоритетов попадают в выбранный диапазон.

Главная польза: план перестает быть отдельной таблицей “для просмотра” и становится общей рабочей картиной. Один сотрудник двигает план, другой отмечает свой этап, третий видит, что можно делать дальше.

Для чтения

A.6 Гость смотрит систему

Лучший канал: веб.

1. Войти под гостем. 2. Открыть журнал. 3. Открыть карточки. 4. Скачать доступные вложения. 5. Открыть таймлайн. 6. Открыть **Документохранилище**. 7. В **Общие документы** скачать нужные файлы.

Гость ничего не меняет.

Читать всем

A.7 Быстро найти файл через Telegram

Лучший канал: Telegram.

1. Написать боту **найди файлы ЖА-43**. 2. Выбрать нужный файл, если вариантов несколько. 3. Скачать файл из ответа.

Администратору

A.8 Администратор заводит нового сотрудника

Лучший канал: веб.

1. Открыть профиль/пользователей. 2. Создать пользователя. 3. Указать ФИО. 4. Указать должность. 5. Выбрать роль. 6. Выдать логин и пароль. 7. Проверить первый вход вместе с человеком.

Готовый текст:

Андрей, привет.

Вот ссылка на Foreman: <https://claim-bot-tzertis.amvera.io/foreman/>

Логин: ...

Пароль: ...

Сначала просто зайти, открой журнал и попробуй создать черновик.

Главный смысл: заявку не нужно вечером собирать в Word. Ее можно сделать сразу в Foreman, сохранить черновик, прикрепить файлы и видеть статус.

Первую заявку можем пройти вместе.

Приложение В. Частые проблемы и решения

В.1 Белый экран в браузере

Порядок:

1. Обновить страницу браузера. 2. Проверить интернет. 3. Открыть адрес заново: <https://claim-bot-tzertis.amvera.io/foreman/> 4. Если не помогло - выйти и войти снова. 5. Если сайт вообще не грузится - проверить состояние сервера.

Администратору:

- проверить `/api/health` ;
- проверить `/healthz` ;
- посмотреть логи Amvera;
- проверить последний деплой.

В.2 Приходится снова вводить пароль

Возможные причины:

- нажали `Выйти` ;
- прошло около 30 дней;
- браузер очистил данные;
- приложение переустановлено;
- старый токен был выдан до продления сессии;
- пароль был изменен;
- аккаунт отключен.

Решение:

1. Войти заново. 2. Если повторяется каждый день, сообщить администратору устройство, браузер и время.

В.3 Неактивная кнопка

Причины:

- нет прав;
- не тот статус заявки;
- не выбран файл;
- не выбрана заявка;

- пользователь гость;
- идет загрузка.

Проверить:

1. Что выбрано нужное количество файлов. 2. Что открыт не гостевой аккаунт. 3. Что заявка является черновиком, если нужно редактирование. 4. Что заполнены обязательные поля.

Для чтения

V.4 Файл прикрепился некрасиво или не видно название

Нужно сообщить:

- номер заявки;
- название файла;
- где прикрепляли: **С компьютера** или **Из документов** ;
- браузер;
- скриншот.

Нормальный вид должен показывать тип, название, размер и кнопку скачивания.

Читать всем

V.5 Мобильная заявка не отправляется

Проверить:

1. Есть интернет. 2. Заполнены объект и номер. 3. Черновик сохранен. 4. Есть e-mail адресат. 5. Нет ошибки **Сначала создай заявку** .

Если связь плохая, сохранить черновик и повторить позже.

Читать всем

V.6 Telegram-бот отвечает не туда

Порядок:

1. Написать **выход** . 2. Написать **меню** . 3. Перейти в нужный раздел кнопкой. 4. Использовать короткую команду.

Приложение С. Администраторская памятка перед внедрением

Перед тем как давать систему сотрудникам, проверить:

Проверка	Где
Веб открывается	Foreman Web
Админ входит	Foreman Web
Обычный пользователь входит	Foreman Web
Гость входит	Foreman Web
Журнал открывается	Foreman Web
Карточка заявки открывается	Foreman Web
Вложения скачиваются	Foreman Web
Файл добавляется с компьютера	Foreman Web
Файл добавляется из документов	Foreman Web
Документохранилище открывается	Foreman Web
Сортировка работает	Foreman Web
Таймлайн показывает 11/2	Foreman Web
PDF таймлайна скачивается	Foreman Web
План-график открывается	Foreman Web
Карточка план-графика переносится внутри объекта	Foreman Web
Запрещенный статус план-графика не проставляется	Foreman Web
Мобильный вход работает	Foreman Mobile
Мобильный черновик сохраняется	Foreman Mobile
Telegram меню работает	Telegram
Telegram мои заявки работает	Telegram
Telegram /status работает для админа	Telegram

Приложение D. Как объяснить сотруднику без давления

Плохой подход:

Все, работаем по-новому, разбирайтесь.

Хороший подход:

Смысл Foreman не в том, чтобы усложнить тебе работу.
Наоборот, он убирает вечернюю ручную возню с Word.

Ты можешь сделать заявку сразу, сохранить черновик, прикрепить фото и документы.
Офис увидит заявку в системе, а не будет ждать отдельное письмо.
Если отвлекли - черновик останется. Если нужен файл - он рядом с заявкой.

Первую заявку пройдем вместе, дальше станет быстрее.

Для пожилого или уставшего человека главное:

- не перегружать терминами;
- показать одну заявку руками;
- дать логин и пароль на бумаге или в личном сообщении;
- объяснить, что Сохранить черновик - это нормально;
- не требовать сразу знать все разделы.

Администратору

Приложение Е. Что считать главным документом

Главный документ по системе:

```
E:\файлы к чат-боту\Программа_для прорабов_Рекламации\ЕДИНОЕ_РУКОВОДСТВО_FOREMAN.md
```

README Telegram-бота остается подробной справкой именно по Telegram-боту. Он не заменяет это единое руководство.

Если меняется функциональность веба, мобильного приложения или бота, сначала обновляется это руководство, затем при необходимости уточняются отдельные технические README.